

Causal Search for Skylines (CSS): Causally-Informed Selective Data De-Correlation

Pratanu Mandal
pmandal5@asu.edu
Arizona State University
Tempe, Arizona, USA

K. Selçuk Candan
candan@asu.edu
Arizona State University
Tempe, Arizona, USA

Abhinav Gorantla
agorant2@asu.edu
Arizona State University
Tempe, Arizona, USA

Maria Luisa Sapino
mlsapino@di.unito.it
University of Turin
Turin, Italy

Abstract

Skyline queries are popular and effective tools in multi-criteria decision support as they extract *interesting* (pareto-optimal) points that help summarize the available data with respect to a given set of preference attributes. Unfortunately, the efficiency of the skyline algorithms depends heavily on the underlying data statistics. In this paper, we argue that the efficiency of the skyline algorithms could be significantly boosted if one could erase any attribute correlations that do not agree with the preference criteria, while preserving (or even boosting) correlations that agree with the user provided criteria. Therefore, we propose a *causally-informed selective de-correlation* mechanism to enable skyline algorithms to better leverage the pruning opportunities provided by the positively-aligned data distributions, without having to suffer from the mis-alignments. In particular, we show that, given a causal graph that describes the underlying causal structure of the data, one can identify a subset of the attributes that can be used to selectively de-correlate the preference attributes. Importantly, the proposed *causal search for skylines* (CSS) approach is agnostic to the underlying candidate enumeration and pruning strategies and, therefore, can be leveraged to improve any popular skyline discovery algorithm. Experiments on multiple real and synthetic data sets and for different skyline discovery algorithms show that the proposed causally-informed selective de-correlation technique significantly reduces both the number of dominance checks as well as the overall time needed to locate skyline points.

1 Introduction

Skyline queries are popular and effective tools in decision support. Intuitively, a skyline query extracts *interesting* (pareto-optimal) points that help summarize the available data question with respect to a given set of preference attributes, providing insights into the diversity of the data across these attributes. Given a set, D , of data points in an attribute/feature¹ space, A , and a set, $P \subseteq A$ of preference attributes, the *skyline* of D consists of the subset of points that are not dominated² by any other data point in D [1]. Searching for such non-dominated data is valuable in many applications that involve multi-criteria decision making [2].

¹We use the terms "attribute", "feature", and "variable" interchangeably.

²A point dominates another point if it is as good or better in all preference attributes, and better in at least one attribute.

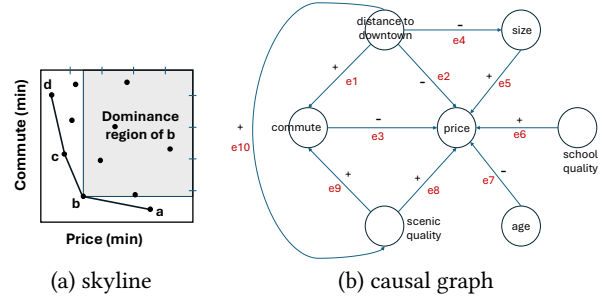


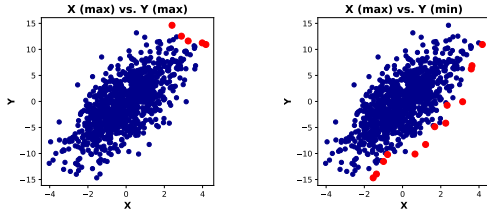
Figure 1: Running example: skylines for house hunting

EXAMPLE 1 (HOUSE HUNTING). Let us consider a user who aims to identify a subset of homes in the market with low Price and low Commute. This task can be formulated as searching a non-dominated subset of houses in the market with respect to the preference attributes, $P = \{\text{Price}, \text{Commute}\}$ and the corresponding preference criteria, $\Theta = \{\theta_{\text{Price}}, \theta_{\text{Commute}}\}$, where $\theta_{\text{Price}} = \min$ and $\theta_{\text{Commute}} = \min$. $\circ$

Based on these specifications, any house that is in the skyline subset will either have a lower Price or lower Commute than all the other houses in the data set; in other words, the user cannot find any houses that are not in the skyline but have both lower Price and lower Commute than the houses in the skyline. In Figure 1(a), the shaded area indicates the dominance region of the house b : for any house in this preference range, the house b is either cheaper or has lesser commute. Therefore, b can be said to be more *useful* (with respect to the preference criteria) than all houses that it dominates.

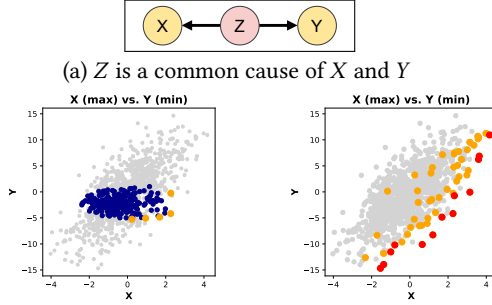
1.1 Challenge: Impact of the Data Distribution on Skyline Discovery Performance

As we will further discuss in Section 2, in the literature there are several algorithms developed for identifying skyline data sets. These include nested-loop skylines [1], sort-first skylines [3], and index-supported skylines [4]. Intuitively, all of these algorithms rely on an enumerate-and-prune strategy: they enumerate candidate skyline objects in the data set and they use dominance-checks to prune away those objects that should not be included in the skyline set. The primary goal of these algorithms is to enumerate the skyline candidates in such a way that, the data set can be pruned away in the most efficient manner (i.e., using the fewest dominance checks and with the least amount of memory to store the candidate objects). Unfortunately, the efficiency of the skyline algorithms depends heavily



(a) max-max with '+' corr (b) max-min with '+' corr

Figure 2: Impact of the alignment between data distribution and preference criteria: in (a) the dominance region is determined by a few extreme points, whereas in (b) there are a large number of skyline points on the Pareto front



(b) skyline of one cluster of Z (c) skyline of skylines

Figure 3: Impact of clustering on the (a) confounder Z: (b) within each cluster, the correlation between X and Y is close to zero, and (c) seeking the skyline among the merged Pareto fronts (orange dots) across clusters is much less expensive, as many tuples have already been pruned

on the underlying data statistics: as illustrated in Figure 2, for data sets where the attribute correlations mirror the corresponding preference criteria (positively-correlated attributes for max-max and min-min preference criteria and negatively-correlated for max-min and min-max criteria), the number of skyline objects are fewer; whereas for data where the attribute correlations are mis-aligned with the preference criteria, the skyline set may contain large numbers of non-dominated data objects.

More critically though, when data correlation is mis-aligned with the preference criteria, it becomes especially difficult to enumerate the data objects in a way that helps prune non-skyline objects without having to rely on large numbers of explicit dominance checks. As a result, the cost of enumerating skylines explodes in situations where there is a mis-alignment between the preference criteria and the underlying data distribution. In this paper, we point out that we can potentially exploit underlying causal structures in the data to optimize skyline computation. In Figure 3(a), attribute Z causally influences both X and Y , creating an apparent correlation between them and, since the preference criterion is max-min, the resulting misalignment would render the skyline search expensive as we see in Figure 2(b). As we discuss in Section 3.2, clustering on Z would reduce the correlation between X and Y within each cluster to approximately zero, making the skyline computation for each cluster much less expensive (Figure 3(b)). If we then merge the resulting per-cluster pareto fronts and seek the skyline of the

merged set, since the number of considered tuples in the merged set is far smaller than in the original data set, the negative impact of misalignment on the cost of the skyline search can be significantly reduced (Figure 3(c)).

1.2 Our Contributions: Causally-Informed Selective Data De-Correlation

As outlined above, in this paper, we argue that the efficiency of the skyline algorithms could be significantly boosted if one could erase any attribute correlations that do not agree with the preference criteria (i.e., *negatively-aligned* data distributions), while preserving (or even boosting) correlations that agree with the user provided criteria (i.e., *positively-aligned* data distributions). If such **selective de-correlation** were possible, the skyline algorithms would benefit from the effective pruning opportunities provided by the positively-aligned data distributions, without having to suffer from the mis-alignments. In the rest of the paper, we show that this challenge can be tackled effectively in situations where we have *a priori* causal-knowledge³ that explains the underlying data distributions. In particular,

- we show that, given a causal graph that describes the underlying causal structure of the data, one can identify a subset, Z , of the attributes that can be used to selectively de-correlate the preference attributes and
- since positively-aligned attribute pairs are desirable for efficient skyline discovery, whereas negatively-aligned attribute pairs need to be avoided, we present an algorithm to select a subset, Z^+ , of attributes that can be used to de-correlate the negatively-aligned preference attributes.

Importantly, the proposed *causal search for skylines (CSS)* approach is agnostic to the underlying candidate enumeration and pruning strategies and, therefore, can be leveraged to improve any popular skyline discovery algorithm, including (but not limited to) nested-loop skylines [1], sort-first skylines [3], index-supported skylines [4], as well as more advanced skyline algorithms, such as k -dominant skylines [8]. In Section 7, we experimentally evaluate the proposed *causal-skyline* approach on multiple real and synthetic data sets and for different skyline discovery algorithms. Experiment results show that the proposed causally-informed selective de-correlation technique significantly reduces both the number of dominance checks as well as the time needed to obtain the skyline.

2 Related Works

Skyline queries are commonly used in multi-criteria decision making [9–13]. In [10], authors use skylines to find the best clustering of data. [11] uses skyline queries to generate recommendations based on user preferences. Due to their ability to reduce the volume of data within the context of a user provided preference criterion, the task of processing skyline queries in an efficient manner has attracted considerable attention. The task of finding the non-dominated set of data points was attempted by Kung *et al.* in 1975 under the name of the maximum vector problem [14]. The algorithm proposed in [14] is based on the divide and conquer principle. Kung’s

³When such causal knowledge is not available, we can rely on causal discovery algorithms [5–7] to learn causal graphs, as shown in Section 7.2.5.

algorithm lead to the development of various skyline algorithms designed for specific situations, including those devised for static environments [1, 15], high dimensional data sets [16] and parallel environments [17–20]. It also inspired novel skyline algorithms for progressive skyline queries [4], online processing [21–23], skyline-join queries [15, 24, 25] and data streams [26–28].

Borzsonyi *et al.* [1] were the first to coin and investigate the *skyline* computation problem in the context of databases. The authors extended Kung’s divide and conquer algorithm so that it works well on large databases. They proposed a *Block-Nested-Loops (BNL)* algorithm, which keeps an in-memory window of incomparable points and reports a point as a skyline result only if it is not dominated by any other point in the database. They also proposed a *divide and conquer* based algorithm, which divides the data space into several regions, calculates the skyline in each region, and produces the final skyline from the points in the regional skylines.

The *Sort-Filter-Skyline (SFS)* algorithm [3], which is based on the same principle as the BNL algorithm, improves on performance by first sorting the data according to a monotone function. Bartolini *et al.* also developed a sort-based skyline technique called the *Sort and Limit Skyline algorithm (SaLSa)* that uses the idea of presorting input tuples to limit the number of tuples read and compared during skyline query processing [29]. Tan *et al.* proposed progressive skyline algorithms called *bitmap* and *index* [4]. The *bitmap* method is completely non-blocking and exploits a bitmap structure to quickly identify whether a point is a skyline result or not. The *index* method, on the other hand, exploits a transformation mechanism and a B^+ -tree index to return skyline points in batches. Other contributions to skyline query processing include online algorithms, such as [21] and [22], based on nearest-neighbor search. Bentley *et al.* developed the *Fast Linear Expected Time (FLET)* algorithm [30] which improved upon Kung’s divide and conquer based algorithm by probabilistically eliminating points which cannot be part of the skyline. Godfrey *et al.* [31] demonstrated that the performance of Kung’s divide and conquer based approach is poor with respect to the dimensionality of the problem. They also introduced the *Linear Elimination Sort for Skyline (LESS)* algorithm which combines aspects of BNL, SFS, and FLET, without any aspects of divide and conquer approach.

Park *et al.* proposed a mechanism for parallel processing of skyline queries to exploit multicore processors [32]; the authors also proposed a parallel version of the BBS algorithm. Lee *et al.* [33] proposed an index structure called *ZBtree* to index and store data points based on the Z-order curve. They also developed several skyline algorithms that utilize the *ZBtree* index to efficiently process skyline queries. Huang *et al.* [20] also proposed a parallel skyline algorithm for multi-processor clusters that utilizes Z-order clustering to reduce dominance checks. Shang *et al.* examined the skyline operator in the context of anti-correlated distributions [34]. The authors proposed a probabilistic cardinality model for anti-correlated distributions and analyzed the upper and lower bounds of the expected value of skyline cardinality. They also developed an algorithm called *SOAD (Skyline Operator on Anti-correlated Distributions)* that effectively eliminates non-promising points based on a *determination and elimination* framework.

There has also been significant work on estimating the cardinality of skyline sets. [35] analyzes the distribution of admissible

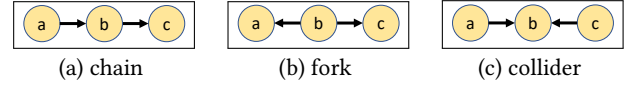


Figure 4: Three basic causal structures

pareto-optimal points and [36] investigates the expected number of maximal vectors in a set, providing insights into the average-case of skyline cardinality. [37, 38] proposed skyline cardinality estimation methods using log-sampling (LS). [38] introduced a non-parametric kernel-based (KB) skyline cardinality estimation technique which improves upon the log-sampling (LS) method and does not depend on the assumption that the preference attributes have to be independent of each other. [39] introduces a so-called purely-sampling based approach (PS), that does not involve complex mathematical computations and, hence is more efficient than KB. One of the more recent works in skyline cardinality estimation is [40], which outperforms the previous state-of-the-art including [37–39].

3 Preliminaries

This section introduces the key concepts and provides preliminary formalisms we will utilize in the rest of the paper. We assume that the reader is familiar with *basic* concepts in skylines (such as tuple dominance) and causal graphs (such as a causal path and a confounder). Appendix A provides a quick refresher.

3.1 Causal Graphs and Skylines

Causality is the relationship between the effect (outcome) and cause (treatment) that gives rise to it [41] and is a topic of increasing interest in big data analysis [42, 43], machine learning [44] [45], and reinforcement learning [46]. Among others [47–49], Pearl has shown that a priori knowledge, in the form of *causal graphs*, is critical in analyzing and managing data [50].

DEFINITION 1 (CAUSAL GRAPH). Let D be a data set defined in an attribute space, A . A causal graph, $G_A = (V_A, E_A, \lambda)$, corresponding to the attribute set A is an edge-labeled directed acyclic graph (DAG), where for each attribute $a_i \in A$, there is a vertex $v_i \in V_A$ and each edge $e_h = \langle a_i, a_j \rangle$ (or $a_i \rightarrow a_j$) indicates a known direct causal relationship between the attributes a_i and a_j and the edge label $\lambda(e_h)$ indicates the nature of the causal relationship between the two attributes.

Figure 4 presents three basic causal structures introduced in [50]: (a) a *chain* structure where a causally affects c through its influence on b (here b is referred to as a mediator); (b) a *fork* structure where b is the common cause of both a and c (note that, in this case, a and c are likely dependent but there is no causation between them); (c) a *collider* structure where both a and c independently cause b . In this paper, we note that, in the context of skylines, these causal structures underlying the data may introduce statistical dependence among preference attributes:

EXAMPLE 2 (WEAKENING OR STRENGTHENING OF RELATIONSHIPS AMONG PREFERENCE ATTRIBUTES). Let us reconsider our house hunting running example, with preference attributes *Commute* and *Price*. Let us further assume that the causal graph underlying the data is as depicted in Figure 1(b). As we see in this example, there is a direct (negative) causal relationship between the preference attributes, *Commute* and *Price* – more commute generally translates into lower

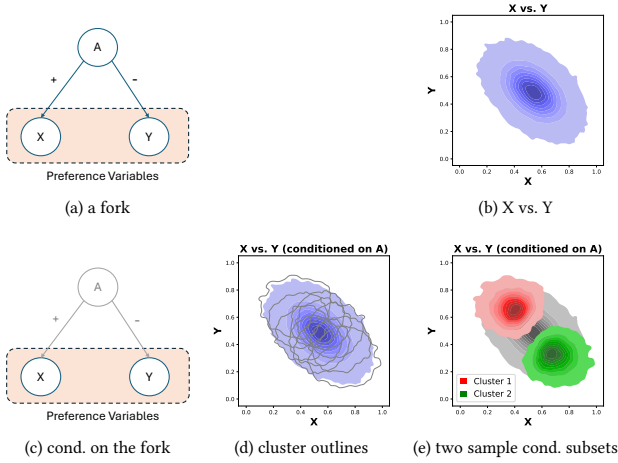


Figure 5: (a,b) The fork attribute A imposes negative correlation on attributes X and Y ; (c,d) conditioning of the attribute A creates multiple clusters of data points, each cluster lacking any negative correlation; in (e), we highlight two of these resulting clusters in red and green

house prices. In addition, there are also several indirect contributors to the relationship between these two preference attributes:

- The `Distance_to_city_center` attribute has a direct causal impact on both `Price` and `Commute` attributes (and therefore is a confounder): if we consider a city (like London), where house prices in the city center are higher due to housing density and that most of the commutable jobs are also located in the city center, the `Distance_to_city_center` attribute would contribute additional negative correlation, strengthening the existing negative correlation between the `Price` and `Commute` attributes.
- In contrast, the `Scenic_quality` attribute impacts both `Commute` and `Price` positively as houses with scenic views generally have longer commutes and they also tend to be more expensive than the other houses. Note that this attribute weakens the negative correlation between the two preference attributes. $\circ$

3.2 Conditioning and De-Correlation

Conditioning is the act of clustering the data entries that share the same value for a given (set of) attribute(s).

DEFINITION 2 (CONDITIONING). Let D be a data set and let $a \in A$ be an attribute. Let $\text{Dom}(a)$ be the domain of this attribute.

$$\text{cond}(D, a) = \{(v, D_v) \mid (v \in \text{Dom}(a)) \wedge (t \in D) \wedge (D_v = \{t.a = v\})\}. \quad \diamond$$

In SQL, this corresponds to a `GROUP BY` operation.

In this paper, we note that the process of conditioning can have impact on the statistical distribution of the values for the remaining attributes. For instance in the *chain* and *fork* structures conditioning on A can eliminate any statistical dependence between X and Y (this is visualized in Figure 5). In contrast, in the *collider* structure conditioning on A can introduce a (negative) correlation between unrelated variables, X and Y (this is visualized in Figure 6).

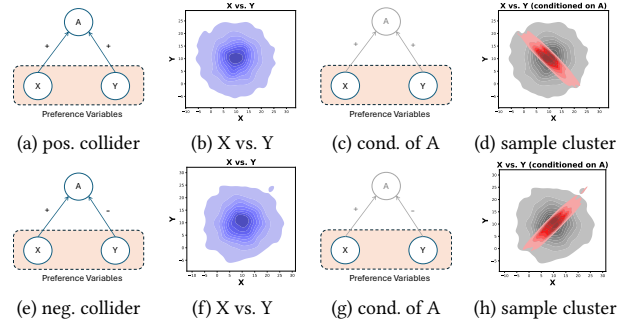


Figure 6: The effect of the conditioning of a collider attribute on the distribution of the data: (a,b,e,f) X and Y are originally independent from each other; (c,d,g,h) conditioning of the collider attribute A , however, creates clusters of data points, each cluster having correlation with a sign opposite of the overall sign of the collision path

KEY OBSERVATION 1 (SELECTIVE DE-CORRELATION). As a corollary to the above, we note that negative correlations can be eliminated by conditioning on the mediator/fork attributes with originally negative impact; similarly, positive correlations can be boosted by conditioning on the colliders which have originally negative impact.

Consider the following example:

EXAMPLE 3 (CONDITIONING TO WEAKEN NEGATIVE CORRELATIONS AMONG THE PREFERENCE ATTRIBUTES). In Example 1, if we condition on the `Distance_to_city_center` by clustering all houses based on their distance to the city center and on whether they have scenic views or not, then within each such cluster, the only remaining causal relationship between `Price` and `Commute` attributes would be the direct causal relationship of `Commute` on `Price`. $\circ$

4 Causal Search for Skylines (CSS) with Selective De-Correlation

Let us reconsider our house hunting example and the corresponding causal graph visualized in Figure 1. In this example, our preference attributes are `Commute` and `Price`, and our preference criteria are min-min. As we have discussed in the previous section, in this graph, we have two confounders `Distance_to_city_center` and `Scenic_view`, the former introducing a negative correlation between the preference attributes, whereas the latter introducing positive correlation between them. While the negative correlation between `Commute` and `Price` due to the “-” labeled direct causal edge between them cannot be eliminated, the additional correlations due to the existence of the confounders can potentially be eliminated by conditioning them:

- The negative correlation imposed on the preference attributes by the `Distance_to_city_center` confounder can be eliminated by conditioning this attribute. Our intuition is that, since negative correlations are mis-aligned with the min-min preference criteria for the `Commute` and `Price` skyline attributes, this should help reduce the number of dominance checks and improve the efficiency of the skyline search process.

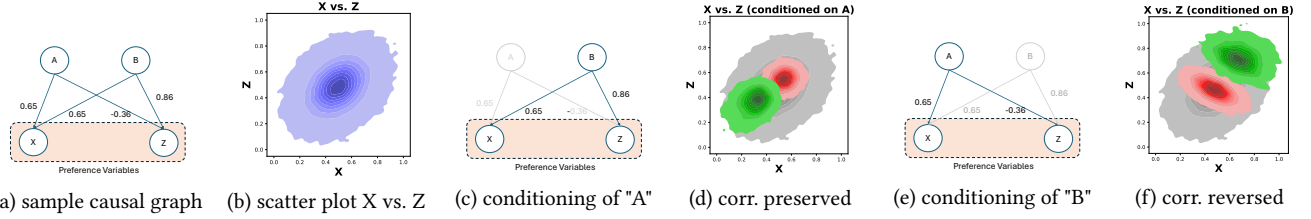


Figure 7: (a,b) A sample causal graph and the corresponding preference data, (c,d) conditioning of "A" preserves the correlations among the preference variables, while (e,f) conditioning of "B" reverses the correlations among the preference variables

- In contrast, however, conditioning of the Scenic_view attribute would eliminate a degree of positive correlation between our preference attributes. This clearly is not desirable when our preference criteria is min-min as the elimination of this positive correlation would effectively emphasize the negative correlation between the preference attributes due to the "-" labeled direct causal edge between them.

Therefore, while conditioning of the Distance_to_city_center confounder is desirable, conditioning of the Scenic_view attribute is not. Note that this indicates a potential approach to improve the efficiency of skyline discovery, when there are only two preference attributes: (1) identify the confounders; (2) identify if the statistical correlation implied by the confounder is positively or negatively aligned with the preference criteria; and (3) condition those confounders that have negative alignment with the preferences to selectively de-correlate the preference attributes. While this approach promises an effective way to reduce the number of dominance checks, there are several challenges:

- Firstly, we do not always have only two preference attributes. We therefore need an efficient and effective way to account for attributes that are negatively aligned with the preference attributes.
- Secondly, data attributes may have partial causal impact on the set of preference attributes and they may be positively-aligned with some of the criteria, while being negatively aligned with the rest. In Figure 7(e,f), we see a scenario where conditioning on one of the attributes reverses the positive correlation between the preference variables, potentially increasing the cost of the skyline search. We therefore need an effective way to account for such mis-alignments to promote the efficiency of the skyline discovery process, without negatively affecting the pruning of non-promising skyline candidates.

Therefore, we present a *causal search for skylines* (CSS) which, given a causal graph that describes the underlying causal structure of the data, identifies a subset, $\mathcal{Z}^+$, of the data attributes that can be used to de-correlate the negatively-aligned preference attributes (while preserving the correlations among the positively-aligned ones).

4.1 Problem Formulation

Table 3 in the Appendix provides the key notations used throughout the paper. Let $R(A)$ be a relation with the attribute set A and let $P \subseteq A$ denote the set of preference/skyline attributes. Let us consider a skyline setting where all preferences are *max* or all of them are *min* (other scenarios can easily be converted to these with simple data transformations). Let G_R (or G for short) be an $\pm$ -edge-labeled causal

graph for the attributes in R . Let further C_P denote a correlation matrix for the attributes in P .

Our goal is to find a subset of conditioning attributes, $\mathcal{Z} \subseteq A$, such that for all $a_i, a_j \in P$, $C_P[i, j] \leq C_P^*[i, j]$, where C_P^* is the correlation matrix obtained after the conditioning on attributes in $\mathcal{Z}$. It is, however, especially important that

- if $C_P[i, j] \ll 0$, then we have $C_P^*[i, j] \gtrsim 0$ and
- if $C_P[i, j] > 0$, then we do not have $C_P^*[i, j] \ll C_P[i, j]$.

More specifically, when the correlation between two preference attributes, i, j , is significantly negatively aligned with the preference criteria ($C_P[i, j] \ll 0$), we would like the correlation between preference variates i, j to be positive, but close to zero ($C_P[i, j] \gtrsim 0$). If, on the other hand, the correlation between a pair of preference attributes, i, j , is positively aligned with the preference criteria ($C_P[i, j] > 0$), then we do not want the correlation to have a significantly drop (i.e., $C_P^*[i, j] \ll C_P[i, j]$) after we perform conditioning.

4.2 Baseline Algorithm - Algorithm #0: Data Driven Conditioning Set Selection

Before discussing algorithms that rely on causal graphs for identifying the conditioning set, we first present a purely data-driven approach to conditioning set selection.

- (1) **Conditioning set enumeration.** Let $\mathcal{Z} \subseteq A$ be a subset of the attribute set of the given relation.
 - (a) *Data conditioning.* Using a GROUP BY operation, the input data set, D , is grouped into multiple subsets, $D_1(\mathcal{Z}), \dots, D_m(\mathcal{Z})$ (where m is the number of unique combinations for the conditioning set of attributes).
 - (b) *Pairwise correlation computation.* We consider all pairs of $a_i, a_j \in P$ for group $D_k(\mathcal{Z})$ to compute the corresponding correlation $C_{\mathcal{Z},P}[i, j, k]$ under the given conditioning set.
 - (c) *Pairwise correlation gain computation.* We then compute the corresponding correlation gain for the given conditioning set, where $C_P[i, j]$ is the correlation without any conditioning.

$$C_{gain}(\mathcal{Z}) = C_{avg}(\mathcal{Z}) - \sum_{i \neq j} C_P[i, j],$$

where

$$C_{avg}(\mathcal{Z}) = \frac{1}{k} \sum_k \sum_{i \neq j} C_{\mathcal{Z},P}[i, j, k].$$

- (2) **Conditioning set selection.** Given all subsets of the attribute set, we select the subset, $\mathcal{Z}$, that provides the largest value of $C_{gain}(\mathcal{Z})$ as the conditioning set.

Once the above process (Step 0) is completed, we proceed with identifying the skylines for the input data set D as follows:

Step 1 using a GROUP BY operation, the input data set, D , is grouped into multiple subsets, $D_1, \dots, D_m$ (where m is the number of unique combinations for the conditioning set of attributes),

Step 2 candidates skylines are identified for each subset separately,

Step 3 these candidate skylines are merged into M , and

Step 4 the final skyline set is identified by running a skyline algorithm on the merged candidate skyline data set, M .

The intuition behind the above process is that the conditioning set we seek should ideally eliminate the negative correlations among the preference variables, while preserving (or even boosting) positive correlations among them. As we empirically establish in Section 7.2.1, the above (baseline) algorithm may be effective in identifying conditioning subsets; yet, this baseline algorithm is also quite costly as in Step 0 we need to compute pairwise correlations among $|P|$ attributes for $2^{|A|}$ conditioning subsets, which can be achieved at $O(|P|^2 \cdot 2^{|A|} \cdot N)$ time, where N is the size of the input data. In the rest of this paper, we present algorithms to compute conditioning sets relying on causal graphs (when they are available), avoiding the cost of computing pairwise correlations for each potential conditioning subset.

5 Algorithm #1: Gain-based Negative Path Blocking

In this section, we describe an algorithm to address this issue by blocking the negative paths, while preserving the positive ones in the underlying causal graph. Below, we provide the outline of the algorithm and discuss each step in detail:

- (1) **Path counting.** In the first step, we consider all pairs of $a_i, a_j \in P$ and for each such pair, we enumerate the set of paths from a_i to a_j using exhaustive BFS or DFS. Let
 - $Paths_{i,j}$ denote the set of all paths between a_i and a_j ;
 - $Open_{i,j}^+$ denote the set of non-blocked (i.e., collider free) paths on which the count of "-" labeled edges is an even number (this implies that the overall correlation implied by the path is positive), and
 - $Open_{i,j}^-$ denote the set of non-blocked (i.e., collider free) paths on which the count of "-" labeled edges is an odd number (this implies that the correlation implied by the path is negative).
- (2) **Conditioning set enumeration.** Let $\mathcal{Z} \subseteq A$ be a subset of the attribute set of the given relation.
 - (a) Preference pair assessment. For this subset, consider all pairs of $a_i, a_j \in P$
 - (i) *Identify new paths that provisionally open* between a_i and a_j . This can happen if a path that was originally blocked through one or more colliders now gets un-blocked, because each collider on the path (or one of its descendants) gets conditioned. Note that, these paths are only provisionally open, because they may in fact be blocked by other conditioned mediator or fork nodes - we will not know this until they are further confirmed.
 - $Prov_{i,j}^+(\mathcal{Z}) \subseteq Paths_{i,j}$ denotes the set of new paths that (provisionally) open between a_i and a_j by $\mathcal{Z}$, such that

the overall correlation implied by the path is positive and

- $Prov_{i,j}^-(\mathcal{Z}) \subseteq Paths_{i,j}$ denotes the set of new paths that (provisionally) open between a_i and a_j by $\mathcal{Z}$, such that the overall correlation implied by the path is negative.

Note, however, that each un-blocked collider reverses the positive/negative relationship between the end-points. Therefore, whether a provisionally opened path is positive or negative depends on whether the sum of the number of "-" labeled edges and the number of colliders on the path is even or odd.

- (ii) *Revise the sets of open paths with the provisional set.* Given the above, we can revise the sets of open paths between a_i and a_j considering the paths that provisionally open thanks to the un-blocked conditioners in $\mathcal{Z}$:
 - $Open_{i,j}^+(\mathcal{Z}) = Open_{i,j}^+ \cup Prov_{i,j}^+(\mathcal{Z})$, and
 - $Open_{i,j}^-(\mathcal{Z}) = Open_{i,j}^- \cup Prov_{i,j}^-(\mathcal{Z})$.

- (iii) *Identify the paths that get blocked by $\mathcal{Z}$.* Next, we consider the open paths between a_i and a_j and see if any of these get blocked by the conditioning of any mediators or forks:
 - $Blocked_{i,j}^+(\mathcal{Z}) \subseteq Open_{i,j}^+(\mathcal{Z})$ denotes the set of positive paths that were (provisionally) open between a_i and a_j that are blocked by conditioning of $\mathcal{Z}$; and
 - $Blocked_{i,j}^-(\mathcal{Z}) \subseteq Open_{i,j}^-(\mathcal{Z})$ denotes the set of negative paths that were (provisionally) open between a_i and a_j that are blocked by conditioning of $\mathcal{Z}$.

- (iv) *Evaluate the positive and negative path impact of $\mathcal{Z}$,* in terms of the number of positive and negative paths between a_i and a_j that were open at the beginning versus the number of such paths open after the conditioning of $\mathcal{Z}$:
 - $imp_{i,j}^+(\mathcal{Z}) = |Open_{i,j}^+(\mathcal{Z}) \setminus Blocked_{i,j}^+(\mathcal{Z})| - |Open_{i,j}^+|$,
 - $imp_{i,j}^-(\mathcal{Z}) = |Open_{i,j}^-(\mathcal{Z}) \setminus Blocked_{i,j}^-(\mathcal{Z})| - |Open_{i,j}^-|$.

An impact value < 0 indicates a drop in the corresponding open paths, whereas a value > 0 indicates an increase.

- (b) *Compute the overall gain provided by $\mathcal{Z}$.* The overall impact of the conditioning of $\mathcal{Z}$ is a function of the changes in the numbers of positive and negative open paths as well as of the (absolute value) of the correlation between a_i and a_j .

$$gain(\mathcal{Z}) = \sum_{a_i, a_j \in P} [imp_{i,j}^+(\mathcal{Z}) - imp_{i,j}^-(\mathcal{Z})]$$

- (3) **Conditioning set selection.** For conditioning, we select $\mathcal{Z} \subseteq A$ that provides the largest value of $gain(\mathcal{Z})$.

Note that if we are working with graphs where edges are not labeled with $\pm$, we assume that all open paths are negative paths. Note also that, if we have more precise edge impact values associated with the edges, we can also accommodate them. More specifically, we revise the computation of the impact values, $imp_{i,j}^+(\mathcal{Z})$ and $imp_{i,j}^-(\mathcal{Z})$, in Step 2(a)iv of the above algorithm to take into account the causal weights (cw) of the paths:

- $\mathfrak{D}_{i,j}^+ = \sum_{p \in Open_{i,j}^+} cw(p)$,
- $\mathfrak{D}_{i,j}^- = \sum_{p \in Open_{i,j}^-} cw(p)$,
- $\mathfrak{DB}_{i,j}^+(\mathcal{Z}) = \sum_{p \in Open_{i,j}^+(\mathcal{Z}) \setminus Blocked_{i,j}^+(\mathcal{Z})} cw(p)$,
- $\mathfrak{DB}_{i,j}^-(\mathcal{Z}) = \sum_{p \in Open_{i,j}^-(\mathcal{Z}) \setminus Blocked_{i,j}^-(\mathcal{Z})} cw(p)$,
- $imp_{i,j}^+(\mathcal{Z}) = \mathfrak{DB}_{i,j}^+(\mathcal{Z}) - \mathfrak{D}_{i,j}^+$, and

Algorithm 1 Skylines with Gain-based Negative Path Blocking (*gnSky*)

Input $R(A)$, $P \subseteq A$, D
Output Skyline set S , on preference attributes, P

procedure *gnSky*($R(A)$; P , D)
 $\triangleright$ Step 0: Gain-based Negative Path Blocking

$C_P \leftarrow \text{correlation_matrix}(P, D)$
for $a_i, a_j \in P$ **do**
 $(\text{Paths}_{i,j}; \text{Open}_{i,j}^+; \text{Open}_{i,j}^-) \leftarrow \text{paths}(G, a_i, a_j)$
end for
for $Z \subseteq A \setminus P$ **do**
 $I(Z) \leftarrow \emptyset$
 for $a_i, a_j \in P$ **do**
 $(\text{Prov}_{i,j}^+; \text{Prov}_{i,j}^-) \leftarrow \text{prov_open}(G, Z, a_i, a_j)$ $\triangleright$ Below, * stands for + or -
 $\text{Open}_{i,j}^*(Z) \leftarrow \text{Open}_{i,j} \cup \text{Prov}_{i,j}^*(Z)$
 $\text{Blocked}_{i,j}^*(Z) \leftarrow \text{get_blocked}(\text{Open}_{i,j}^*(Z), Z)$
 $\text{imp}_{i,j}^*(Z) \leftarrow \text{compute_impact}(\text{Open}_{i,j}^*(Z), \text{Blocked}_{i,j}^*(Z), \text{Open}_{i,j}^+(Z))$
 $I(Z) \leftarrow I(Z) \cup \text{imp}_{i,j}^*(Z)$
 end for
 $\text{gain}(Z) \leftarrow \text{compute_gain}(I(Z), C_P)$
end for
 $Z^- \leftarrow \text{argmax}_Z \text{gain}(Z)$ $\triangleright$ Step 1: Conditioning step

$D_1, \dots, D_m \leftarrow \text{GROUP_BY}_{Z^-}(D)$ $\triangleright$ Step 2: Candidate skylines per conditioning value
 $\forall_i S_i \leftarrow \text{skyline}(P, D_i)$
 $C \leftarrow \bigcup_i S_i$ $\triangleright$ Step 3: Merging of candidate skylines
 $S \leftarrow \text{skyline}(P, C)$ $\triangleright$ Step 4: Computing of the final skylines
end procedure

- $\text{imp}_{i,j}^-(Z) = -(\odot B_{i,j}^-(Z) - \odot_{i,j}^-)$,

where, given a path p , the causal weight $\text{cw}(p)$ of the path is determined as the multiplication of the causal edges on the causal path; i.e, $\text{cw}(p) = \prod_{e \in p} \text{cw}(e)$.

This gives us the following *gain-based negative path blocking causal skylines (gnSky)* algorithm: Once the conditioning set is selected based on gain as described above (let us refer to the above process as *Step 0*), we proceed to identify the skylines for the input data set D using the same 4 steps (Steps 1 through 4) in the baseline algorithm (Section 4.2). The outline of the overall algorithm is presented in Algorithm 1. Note that the effectiveness of the algorithm depends on, among other things, the cost of identifying candidate skylines for the subsets in *Step 2* (which is a function of the number and size of the groups and the correlation within each group) and the size of the merged candidate skyline set obtained in *Step 3*. Therefore, as we discuss in the next section, the grouping operation in *Step 1* is critical in the overall performance of causal skyline search.

6 Algorithm #2: Leaky Negative Path Blocking

While the algorithm presented in the previous section helps eliminate negative correlations by blocking paths on the causal graph that lead to negative correlations among the preference attributes, this algorithm suffers from several shortcomings as outlined below.

6.1 Issue #1: Leaky Blocking

As we have discussed in Section 3.2, the GROUP BY operation is analogous to conditioning and the algorithm presented in the previous subsection leverages this to block (for mediator and fork structures)

or open (for colliders) paths on a causal graph to eliminate negative correlations among the preference attributes, while maintaining the positive correlations. The problem, of course, is that in practice grouping the input tuples by a single value combination of the conditioning attributes may be overly selective – it is possible that, the GROUP BY operation returns a single tuple per each unique value combination of the conditioning set of attributes; in such a case, we would have $M = D$ and the cost of the *Step 4* of the algorithm above would be equal to the cost of running the skyline on the original data, rendering the total cost of *Steps 1 through 3* a redundant overhead. In order to avoid this outcome, we can revise the *Step 1* of the algorithm above as follows:

*Step 1** given the target number, m , of groups, use a clustering algorithm to group the input data set, D , into multiple subsets, $D_1, \dots, D_m$.

On the positive side, this revision enables us to control the number of groups that are created in *Step 1*, enabling us to deal with the situations where the combined selectivity of the conditioning set of attributes is high. On the negative side, however, the replacement of *single-value grouping* in *Step 1* with *similar value grouping* in *Step 1** potentially reduces the effect of conditioning and path blocking. We next illustrate this problem, which we refer to as *leaky conditioning* or *leaky blocking*, with an example:

EXAMPLE 4 (LEAKY CONDITIONING/BLOCKING). *This is visualized in Figure 8, here we see a sample causal graph, which has two alternative conditioning sets, $\{B\}$ and $\{A, B\}$ with the same gain. In this example, the data has been grouped into 10 subsets using K-means clustering and in Figures 8(d) and (f), we are showing two of the resulting clusters for each of the conditioning set scenarios.*

As we see in the figure, when we use the groups formed by the conditioning set, $\{B\}$, the correlation between preference attributes X and Z is brought down from -0.44 for the original data set to ~ -0.15 for the resulting clusters; while this conditioning set blocks the negative path between X and Z , the blockage is imperfect or leaky.

In contrast, when we use the conditioning set, $\{A, B\}$, the correlation between preference attributes X and Z is brought down from -0.44 to ~ 0.04 for the resulting clusters; in other words, this conditioning set blocks the negative path between X and Z very effectively, almost completely eliminating the negative correlation between the preference attributes in the resulting clusters.

Therefore, when we are relying on cluster-based *similar-value grouping* in *Step 1* of the algorithm, rather than *single-value grouping*, the value of *gain* defined in Section 5 is not sufficient to differentiate among the path blocking alternatives. Instead, as we discuss in the next section, we need to expand the definition of *gain* in a way that accounts also for the leakages of the conditioning attributes.

6.2 Issue #2: Noisy Causal Information Passing

Imperfect causal information passing is not limited to only the blocked nodes. Even open nodes may be imperfect conduits of causality. As discussed in Section A.2, a common causal model [51, 52], for example, represents the causal relationship $X \rightarrow Y$ as $Y = c_{xy}X + \epsilon_y$, where c_{xy} denotes the causal impact of X on Y , whereas ϵ_y , the noise that acts on attribute Y along this causal edge. When the noise, $\epsilon_y \sim 0$, the causal impact of X and Y is clear. When

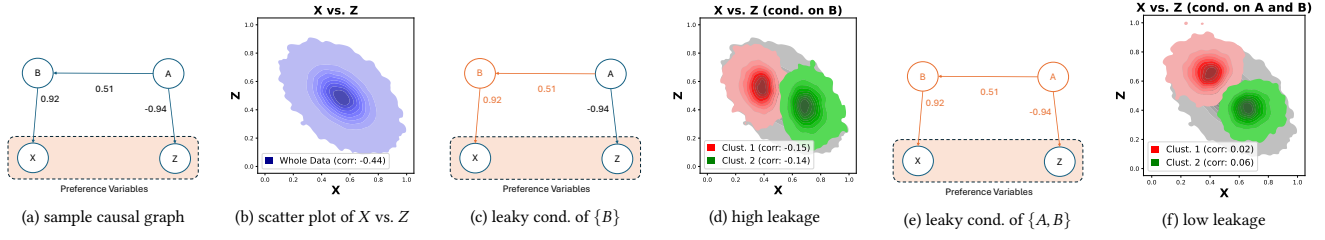


Figure 8: (a) A sample causal graph, (b) scatter plot of two attributes X and Z , (c,d) two sample clusters obtained through high leakage conditioning set $\{B\}$, and (e,f) two sample clusters obtained through low leakage conditioning set $\{A, B\}$ – in the causal graphs, orange colored nodes and edges indicate leaky conditioning/blocking; as we see in this figure, the larger the number of conditioned attributes on a path, the lesser the information leak on that path

$|\epsilon_y| \gg 0$, the causal relationship between X and Y will be noisy and the impact of X on Y may be hidden, even for large values of c_{xy} .

6.3 Leaky Negative Path Blocking

We address leaky blocking and noisy information passing problems by blocking the negative paths, while preserving the positive ones in the underlying causal graph, in a way that takes into account leaky conditioning due to cluster-based *similar-value grouping* in *Step 1* of the algorithm. The algorithm shares the same outline as the *gain-based negative path blocking skyline (gnSky)* algorithm in Section 5, but instead of treating each node in the graph as open or closed, it considers a degree of information passing, π , on a conditioned attribute node: A conventionally open/non-blocked node passes complete information ($\pi = 1$), whereas a conventionally (i.e., *single-value grouping* based) blocked node passes no information at all ($\pi = 0$). In contrast, a *noisy open* node passes $\pi = \lambda_o$ information, whereas a *similar-value grouping* based blocked node passes $\pi = \lambda_b$ information. Here $0 < \lambda_b \ll \lambda_o < 1$ are the corresponding degrees of information passage for imperfectly open and blocked nodes, respectively. The outline of the *leaky negative path blocking skyline (lnSky)* algorithm is as follows:

- (1) **Path counting.** In the first step, we consider all pairs of $a_i, a_j \in P$ and for each such pair, we enumerate the set of paths from a_i to a_j using exhaustive BFS or DFS. Let $Paths_{i,j}$, $Open_{i,j}^+$, and $Open_{i,j}^-$ be defined as before.
- (2) **Conditioning set enumeration.** Let $Z \subseteq A$ be a subset of the attribute set of the given relation.
 - (a) **Preference pair assessment.** For this subset, consider all pairs of $a_i, a_j \in P$
 - (i) **Identify new paths that provisionally open** between a_i and a_j . As before, this can happen if a path that was originally blocked through one or more colliders now gets un-blocked, because each collider on the path (or one of its descendants) gets conditioned. Let $Prov_{i,j}^+(Z)$ and $Prov_{i,j}^-(Z)$ be defined as before.
 Note that, in this case, how much information passes on a path $p \in Prov_{i,j}^+(Z) \cup Prov_{i,j}^-(Z)$ depends on how much information passes on the conditioned colliders.
 - (ii) **Revise the sets of open paths with the provisional set.** Given the above, we can revise the sets of open paths between a_i and a_j considering the paths that provisionally open thanks to the un-blocked conditioners in Z :

- $Open_{i,j}^+(\mathcal{Z}) = Open_{i,j}^+ \cup Prov_{i,j}^+(\mathcal{Z})$, and
 - $Open_{i,j}^-(\mathcal{Z}) = Open_{i,j}^- \cup Prov_{i,j}^-(\mathcal{Z})$.
- (iii) **Identify the paths that get blocked by Z .** Next, we consider the open paths between a_i and a_j and see if any of these get blocked by the conditioning of any mediators or forks:
 - $Blocked_{i,j}^+(\mathcal{Z}) \subseteq Open_{i,j}^+(\mathcal{Z})$ denotes the set of positive paths that were (provisionally) open between a_i and a_j that are blocked by conditioning of Z ; and
 - $Blocked_{i,j}^-(\mathcal{Z}) \subseteq Open_{i,j}^-(\mathcal{Z})$ denotes the set of negative paths that were (provisionally) open between a_i and a_j that are blocked by conditioning of Z .
 In this case, conditioning does not entirely eliminate information passage on the paths, but reduces it due to leakages.
 - (iv) **Compute noise and leakage adjusted causal weights of the paths.** Given an open or closed path p , let the causal weight $cw(p)$ of the path be determined as the multiplication of the weights of the causal edges on the causal path; i.e., $cw(p) = \prod_{e \in p} cw(e)$. Let $o_p(Z)$ be the number of open nodes on p and let $b_p(Z)$ be the number of blocked nodes on p , given Z . The noise and leakage adjusted causal weight of p is computed as

$$c\hat{w}(p) = cw(p) \cdot \lambda_o^{o_p(Z)} \cdot \lambda_b^{b_p(Z)}.$$

- (v) **Evaluate the leaky impact of Z on positive and negative paths** between a_i and a_j , in terms of the amount of information flow on them:
 - $\mathfrak{D}_{i,j}^+ = \sum_{p \in Open_{i,j}^+} c\hat{w}(p)$,
 - $\mathfrak{D}_{i,j}^- = \sum_{p \in Open_{i,j}^-} c\hat{w}(p)$,
 - $\mathfrak{D}_{i,j}^+(\mathcal{Z}) = \sum_{p \in Open_{i,j}^+(\mathcal{Z}) \cup Blocked_{i,j}^+(\mathcal{Z})} c\hat{w}(p)$,
 - $\mathfrak{D}_{i,j}^-(\mathcal{Z}) = \sum_{p \in Open_{i,j}^-(\mathcal{Z}) \cup Blocked_{i,j}^-(\mathcal{Z})} c\hat{w}(p)$,
 - $l_imp_{i,j}^+(\mathcal{Z}) = \mathfrak{D}_{i,j}^+(\mathcal{Z}) - \mathfrak{D}_{i,j}^+$, and
 - $l_imp_{i,j}^-(\mathcal{Z}) = -(\mathfrak{D}_{i,j}^-(\mathcal{Z}) - \mathfrak{D}_{i,j}^-)$.
 Note that, a (leaky) impact value < 0 indicates a drop of information flow on the corresponding open paths, whereas a value > 0 indicates an increase.
- (b) **Compute the overall leaky gain provided by Z .** The overall (leaky) impact of the conditioning of Z is a function of the changes of the amount of positive and negative information

Algorithm 2 Skylines with Leaky Negative Path Blocking (*lnSky*)

Input $R(A)$; $P \subseteq A, D, m$
Output Skyline set S , on preference attributes, P

```

procedure  $\text{lnSky}(R(A), P, D, m)$ 
     $C_P \leftarrow \text{correlation\_matrix}(P, D)$   $\triangleright$  Step 0: Leaky Gain-based Negative Path Blocking
    for  $a_i, a_j \in P$  do
         $(\text{Paths}_{i,j}; \text{Open}_{i,j}^+; \text{Open}_{i,j}^-) \leftarrow \text{paths}(G, a_i, a_j)$ 
    end for
    for  $\mathcal{Z} \subseteq A \setminus P$  do
         $I(\mathcal{Z}) \leftarrow \emptyset$ 
        for  $a_i, a_j \in P$  do
             $(\text{Prov}_{i,j}^+; \text{Prov}_{i,j}^-) \leftarrow$ 
                 $\text{l\_prov\_open}(G, \mathcal{Z}, a_i, a_j)$ 
             $\text{Open}_{i,j}^*(\mathcal{Z}) \leftarrow \text{Open}_{i,j} \cup \text{Prov}_{i,j}^*(\mathcal{Z})$ 
             $\text{Blocked}_{i,j}^*(\mathcal{Z}) \leftarrow \text{get\_l\_blocked}(\mathcal{Z},$ 
                 $\text{Open}_{i,j}^*(\mathcal{Z}))$ 
             $\text{l\_imp}_{i,j}^*(\mathcal{Z}) \leftarrow \text{compute\_l\_impact}(\text{Open}_{i,j}^*(\mathcal{Z}),$ 
                 $\text{Blocked}_{i,j}^*(\mathcal{Z}), \text{Open}_{i,j}^*(\mathcal{Z}))$ 
             $I(\mathcal{Z}) \leftarrow I(\mathcal{Z}) \cup \text{l\_imp}_{i,j}^*(\mathcal{Z})$ 
        end for
         $\text{l\_gain}(\mathcal{Z}) \leftarrow \text{compute\_l\_gain}(I(\mathcal{Z}), C_P)$ 
    end for
     $\mathcal{Z}^- \leftarrow \text{argmax}_{\mathcal{Z}} \text{gain}(\mathcal{Z})$   $\triangleright$  Step 1: Conditioning step
     $D_1, \dots, D_m \leftarrow \text{cluster}(D, \mathcal{Z}^-, m)$ 
     $\forall i, S_i \leftarrow \text{skyline}(P, D_i)$   $\triangleright$  Step 2: Candidate skylines per conditioning value
     $C \leftarrow \bigcup_i S_i$   $\triangleright$  Step 3: Merging of candidate skylines
     $S \leftarrow \text{skyline}(P, C)$   $\triangleright$  Step 4: Computing of the final skylines
end procedure

```

flow as well as of the (absolute value) of the correlation between a_i and a_j .

$$\text{l_gain}(\mathcal{Z}) = \sum_{a_i, a_j \in P} [\text{l_imp}_{i,j}^+(\mathcal{Z}) - \text{l_imp}_{i,j}^-(\mathcal{Z})]$$

(3) **Conditioning set selection.** Given all subsets of the attribute set, we select the one, $\mathcal{Z}$, that provides the largest value of the leaky gain, $\text{l_gain}(\mathcal{Z})$.

Note that, as before, if we are working with graphs where edges are not labeled with $\pm$, we can assume that all open paths are negative paths. Note also that, if we have edge impact values associated with the edges, we can also accommodate them (multiplicatively).

This gives us the following *leaky negative path blocking causal skyline* (*lnSky*) algorithm: Once the conditioning set is selected based on the leaky gain as described above (we refer to the above process as *Step 0* of the *lnSky* algorithm), we proceed to identify the skylines for the input data set D using the same 4 steps in the baseline algorithm (Section 4.2) and the gain-based negative path blocking skyline (*gnSky*) algorithm.

The outline of the algorithm is presented in Algorithm 2.

6.4 Complexity Analysis

Both *gnSky* and *lnSky* algorithms consist of 5 steps (Steps 0 through 4). We analyze the complexity of each step separately:

- **Step 0 - Gain computation and conditioning set selection:** The cost of this step depends on the number of paths considered which is a function of the number of preference variables and the number of paths in the causal graph between pairs of these variables. More specifically, in the worst case, the algorithm would enumerate

$((|P| \cdot |P| - 1)/2) \cdot 2^{|A|}$ number of paths. Each of these paths would then be evaluated for each possible conditioning set. Given the above, the overall worst case cost of the algorithm is $O(|P|^2 \cdot 2^{|A|})$ or equally $O(|P|^2 \cdot 4^{|A|})$.

- **Step 1 - Data conditioning:** GROUP BY would require $O(N)$ time for hashing if the values are categorical or $O(N \log(N))$ time if the values are ordinal and we need to sort the data for similarity based grouping. The cost of the clustering based implementation would depend on m, N , and the clustering algorithm used.
- **Step 2 - Candidate skylines:** Assuming that the data is uniformly divided into m subsets, the cost of this step is $m \cdot \text{cost_skyline}(N/m)$; i.e., $O(m \cdot (N/m)^2)$. Note that, in the worst case (where one of the subsets is almost as large as the original data), of course, the cost would be $\sim \text{cost_skyline}(N)$, which is, generally, $O(N^2)$.
- **Step 3 - Merging of candidate skylines:** Let s_i be the number of skylines returned for the i^{th} subset. The cost of this step is $O(\sum_i s_i)$.
- **Step 4 - Obtaining final skylines:** The cost of this step is $\text{cost_skyline}(\sum_i s_i)$, which is $O((\sum_i s_i)^2)$.

Note that the cost of Step 0 depends on the size and complexity of the causal graph and, while the worst case size is exponential in the number of attributes, in practice, the number of paths that need to be enumerated is much smaller and, as we see in Section 7, the cost of Step 0, (which is independent of the data size, N) is generally negligible relative to the cost of skyline computations, which is $O(N^2)$. Moreover, once the impact values for a causal graph is computed, the gain values can be re-used without any recomputation for a new data set with the same causal structure. Consequently, the overall cost of Steps 1 through 4 depends on (a) how uniformly the data conditioning step splits the data into m subsets and (b) how well the conditioning reduces the number of skyline objects (through elimination of negative correlations among the preference attributes).

6.5 Computation of the Scaling Parameters

The Algorithm #2 (*lnSky*) alleviates the shortcomings of Algorithm #1 (*gnSky*) by accounting for the leaky blocking and noisy causal information passage by leveraging two scaling factors, λ_b and λ_o , respectively. We therefore need a mechanism to quantify these degrees of leakage and noise as a function of the structure of the causal graph. Let p be a path between two preference variables X and Y and $\mathcal{Z}$ be the set of conditioning attributes; to compute the conditioning gain for the path, p , we need to compute two terms:

- $\text{corr}_p(X, Y)$; i.e., the correlation between X and Y , implied by the causal path p , before the conditioning step, and
- $\text{corr}_{p, \mathcal{Z}}(X, Y)$; i.e., the correlation between X and Y , implied by the causal path p , after the conditioning of attributes in $\mathcal{Z}$.

Given these, the conditioning gain for the path given the conditioning set $\mathcal{Z}$ can be computed as $c_gain(p, \mathcal{Z}) = \text{corr}_{p, \mathcal{Z}}(X, Y) - \text{corr}_p(X, Y)$. While outside the scope of this paper, we discuss how to compute these terms, under the assumption that the sets of variables have multi-variate Gaussian distributions, in Appendix C.

Table 1: Experiment parameters (bold indicate default values)

Parameter	Values
Number of data elements	50K, 100K, 200K , 400K
Number of preference attributes	2, 3, 4
Number of de-correlation clusters	2 - 9, 10 , 11 - 20
$\langle \lambda_o, \lambda_b \rangle$ pairs	$\langle 1.0, 0.0 \rangle$, $\langle 0.9, 0.1 \rangle$, $\langle 0.8, 0.2 \rangle$, $\langle 0.7, 0.3 \rangle$, $\langle 0.6, 0.4 \rangle$

Table 2: Real Causal Graphs

Name	# Variates	# Tuples
Abalone [53]	8	4177
Adult [54]	14	48842
Auto MPG [55]	7	398
Concrete Compressive Strength [56]	9	1030
Dry Bean [57]	16	13611
Hong Kong Weather [58]	10	18262
Individual Household Power Consumption [59]	9	2075259
Seoul Bike Demand [60]	13	8760
Wine Quality (White) [61]	11	4898

7 Experiments and Analysis

Next, we present experiment results evaluating the effectiveness of the proposal causally-informed skyline (CSS) search⁴.

7.1 Experiment Setup

7.1.1 Causal Graphs and Data Sets. We consider data sets with both real and synthetic causal graphs for evaluating CSS. For each causal graph, we consider difference preference sets, with varying numbers of preference attributes.

- *Real Causal Graphs:* Table 2 presents the real causal graphs considered in these experiments. Since in this section we also want to investigate the impact of the data size, we varied the data set size for these graphs by leveraging data augmentation using perturbed oversampling with Gaussian noise [62, 63].
- *Synthetic Causal Graphs:* In addition to these real causal graphs, we also considered a number of synthetic causal graphs, with varying complexities (number of nodes, edge densities, and different causal primitives). The causal edge weights for these causal graphs were set to +1 or +0.5 for strongly and weakly positive causal edges, respectively. Similarly, -1.0 and -0.5 were used for strongly and weakly negative causal edges, respectively. The data for these causal graphs were also generated synthetically.
- *Inferred Causal Graphs:* We also experimented with causal graphs inferred from the data, rather than being provided as input by the user. While there are many causal discovery algorithms [5–7], in the experiments, we used the NOTEARS [64] causal inference algorithm, which assumes a simple linear structural equation model, to discover the causal graph and the edge weights.

7.1.2 Skyline Algorithms. As competitors to CSS, we consider the following skyline algorithms:

- *Block Nested Loop Skylines (BNL)* [1] BNL scans the dataset once, keeping a window of currently non-dominated tuples – no specific ordering of tuples is assumed. Every new tuple is compared against this window: the tuple is discarded if it is dominated or inserted into the window, evicting any tuples it dominates.

⁴The source code, along with all causal graphs used in our experiments and their corresponding preference sets and dominance criteria are available at <https://github.com/EmitLab/Causal-Search-for-Skylines>.

- *Sort-First Skylines (SFS)* [3] SFS sorts the data on a monotone ranking criterion ($\langle \text{sum} \rangle$ in the experiments), then performs a scan in that order, maintaining a window of candidates⁵. The presort reduces the number of comparisons.
- *Sort-and-Limit Skyline (SaLSa)* [29] SaLSa builds on SFS by introducing "lazy" sorting: it interleaves (partial) sorting with skyline testing, in such a way that any removed tuple cannot re-enter the skyline. For SaLSa, we have used the $\langle \text{max}, \text{sum} \rangle$ monotonic function as recommended by the authors.
- *Branch and Bound Skyline (BBS)* [22] BBS operates on an R-tree index, exploring nodes in best-first order using a priority queue pruning whole sub-trees if their bounding rectangles are dominated by skyline tuples already found.
- *Divide and Conquer Skylines (D&C)* [1, 14] D&C partitions the space along the median values into sub-regions, computes the skyline of each partition independently, and merges the partial skylines discarding any tuples dominated by another from any partition. The divide-and-conquer strategy of the D&C algorithm can be seen as pure de-correlation of the preference attributes.

7.1.3 CSS Implementation. For each of the above skyline algorithms, we consider the causal version as follows: (a) given a causal graph and the set of preference attributes, we compute the expected gains for each possible conditioning set; (b) given the target number of de-correlation clusters, we apply Euclidean-distance based k -means on the conditioning attributes, normalized to zero mean and unit variance; (c) we compute the causal skylines for each cluster⁶, and (d) obtained skylines from the clusters are unioned and the skyline is searched among these tuples.

Note that the leaky negative path blocking (*lnSky*) algorithm presented in Section 6 uses two parameters $\lambda_b \ll \lambda_o$, denoting the degrees of information passage on imperfectly open and blocked nodes. Table 1 lists the values considered in the experiments.

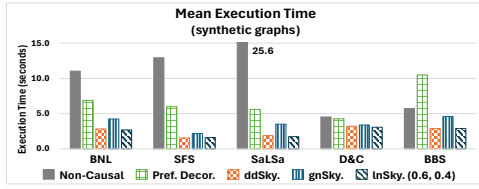
In our implementation, all algorithms use the main memory to compute the skylines, and no temporary file was ever needed (i.e., BNL, SFS, and SaLSa always completed in a single pass). The skyline algorithms, their corresponding causal equivalents, and K-means clustering have been implemented in Python 3.12.7 with Numpy 2.1.2. The experiments were executed using Chameleon [65] on AMD EPYC 7763 64-Core Processor, 256GB RAM, and 500GB SSD.

7.1.4 Baselines. In addition, for each of the above skyline algorithms, we consider the following baselines:

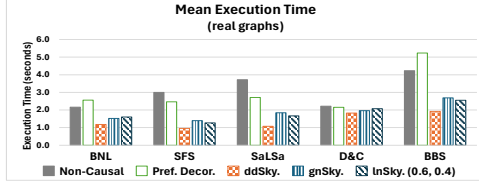
- *Non-causal:* This is the original, non-causal algorithm.
- *Data-driven (ddSky):* Here, we utilize the data-driven strategy to compute the expected gains for each possible conditioning set and subsequently follow the clustering based approach for computing skylines as outlined in the leaky negative path blocking skyline (*lnSky*) algorithm.
- *Preference de-correlation:* In this baseline, we consider a non-selective, preference attribute based de-correlation strategy: we

⁵Although LESS [31] improves upon SFS, we opted for SFS instead due to its lesser complexity and fewer hyper-parameters. In fact, the primary optimization in LESS, the elimination filter (EF), is applied during the external sorting stage, which does not conflict with our proposal.

⁶While both D&C and CSS allow for parallelization, in this section we assume sequential executions of these algorithms.

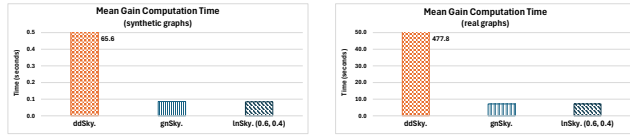


(a) Synthetic causal graphs

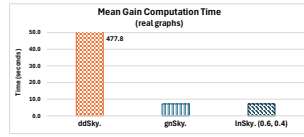


(b) Real-world causal graphs

Figure 9: Cost of skylines with (a) synthetic and (b) real-world causal graphs (the lower, the better) – 200K data



(a) Synthetic causal graphs



(b) Real-world causal graphs

Figure 10: Cost of the expected gain computation step (the lower, the better) – 200K data

apply K-means clustering directly on the set of preference attributes, thereby de-correlating the data without paying attention to whether this would eliminate positive or negative correlations.

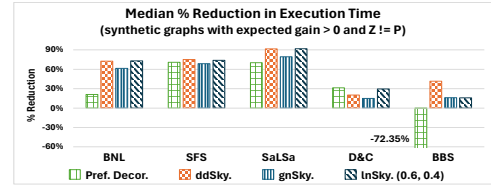
7.1.5 Evaluation Criteria. As evaluation criteria, we consider both (a) the number of dominance checks and (b) execution time of the skyline algorithm (wall clock), which also includes the time to de-correlate the data based on the selected de-correlation attribute set. We separately report the time to compute the de-correlation set. Due to space restrictions, we only report execution times and leave the reporting of dominance checks to Appendix B.

7.2 Experiment Results

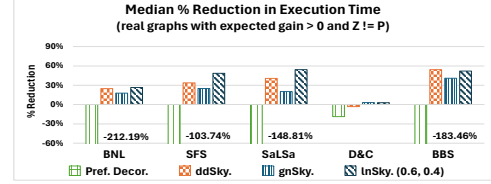
In this section, we present and interpret the experiment results on both synthetic and real-world causal graphs. Importantly, since the divide-and-conquer (D&C) algorithm can be seen as the pure de-correlation of the preference attributes, we distinguish between scenarios where there is a conditioning attribute set predicted to perform better than conditioning on the preference attribute set and the cases where the causal analysis predicts that the preference attribute set itself is the best conditioning attribute set.

7.2.1 Effectiveness of Causal Search for Skylines (CSS).

Overview. The way CSS would be used in practice is to leverage the causal version of the algorithm if there exists a conditioning attribute set with positive predicted gain and revert to the vanilla version otherwise. The benefits of this approach is visualized in Figure 9: As we see in the figure, assuming that the expected gains have already been computed, both *ddSky* and *lnSky* substantially outperform the vanilla versions, indicating that this adaptive strategy

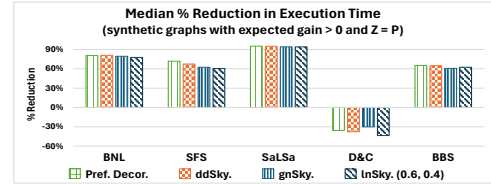


(a) Synthetic causal graphs

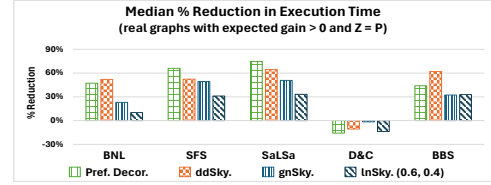


(b) Real-world causal graphs

Figure 11: Median cost reduction (the higher, the better); for these experiments, expected gain is positive and larger than the gain predicted for the preference attribute set (i.e., we expect to see benefits over both vanilla skylines and preference attribute de-correlation) – 200K data



(a) Synthetic causal graphs



(b) Real-world causal graphs

Figure 12: Median cost reduction (the higher, the better); for these experiments, expected gain is positive and equal to the gain predicted for the preference attribute set (i.e., we expect to see benefits over vanilla skylines, but we do not expect to beat preference attribute de-correlation) – 200K data

effectively leverages CSS when it is predicted to be advantageous, while defaulting to the vanilla approach in scenarios where CSS is not expected to be ideal.

Interestingly, when the *lnSky* algorithm is considered, SFS and SaLSa become the fastest algorithms due to the significant reduction of dominance checks from which they benefit, even beating more recent algorithms, such as D&C and BBS.

Cost of the Expected Gain Computation Step. Figure 10 shows the time required to identify the best conditioning attribute set based on different strategies (Step 0). As we expected (see Section 4.2), the gain computation phase of *ddSky* is prohibitively expensive. This prevents it from being applicable in practice, even though it can provide significant gains in terms of execution time as we have already seen in Figure 9. Importantly, the cost of gain

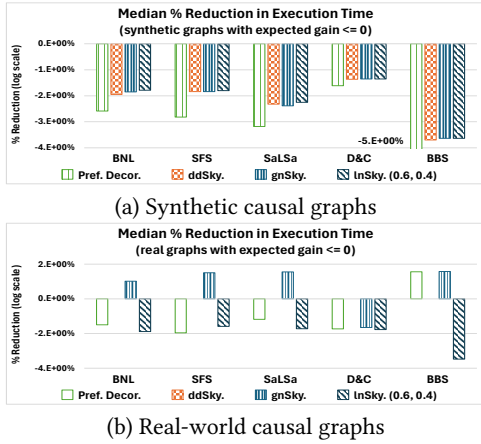


Figure 13: Median cost reduction (the higher, the better); for these experiments, expected gain is negative (i.e., we do not expect to see benefits from de-correlation) – 200K data

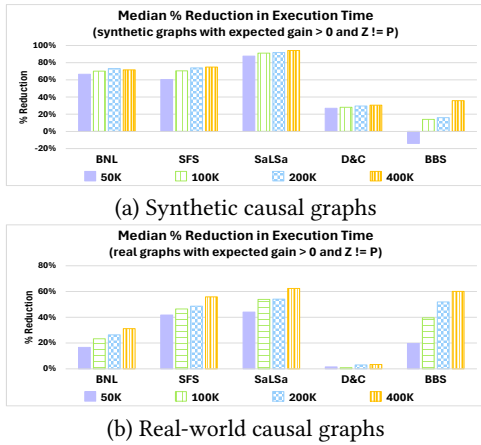


Figure 14: Median cost reduction (the higher, the better) for different numbers of tuples when using $lnSky(0.6, 0.4)$; for these experiments, the expected gain is positive and larger than the gain predicted for the preference attribute set (i.e., we expect to see benefits over both vanilla skylines and preference attribute de-correlation)

computation for $lnSky$ (which is independent of the data size, N) is negligible relative to the cost of skyline computations, making $lnSky$ to be the more practical de-correlation strategy.

Effectiveness of the Gain Prediction. In Figure 9, we have seen that gain prediction can be used to decide whether and how to de-correlate the data to obtain significant reductions in the cost of skyline search. Figure 11 through 13 deep dives in the investigation of the effectiveness of the gain prediction by focusing on the cases where (a) the expected gain is positive and better than that of the preference attributes, (b) gain is positive but not better than that of the preference attributes, and (c) expected gain is negative:

(a) Figure 11 presents the results for scenarios in which there exists a conditioning attribute set with a positive expected gain, predicted to outperform conditioning on the preference attribute set.

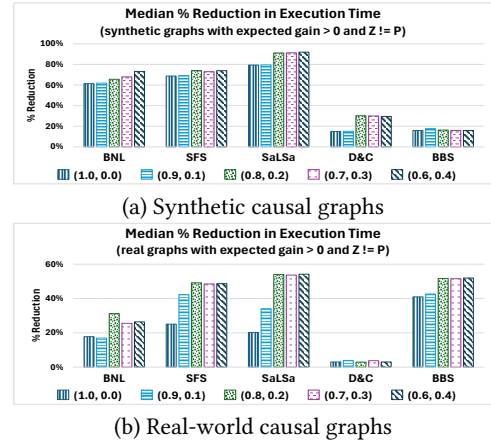


Figure 15: Median cost reduction (the higher, the better) for $\langle \lambda_o, \lambda_b \rangle$ pairs; for these experiments, expected gain is positive and larger than the gain predicted for the preference attribute set (i.e., we expect to see benefits over both vanilla skylines and preference attribute de-correlation) – 200K data

As we see here, in these scenarios, both $ddSky$ and $lnSky$ significantly outperform vanilla (non-causal) baseline algorithms as well as preference-attribute set based de-correlation. Interestingly, in the case of real-world causal graphs, the preference-attribute based de-correlation performs even worse than the vanilla skyline algorithms, while the proposed data- and causality-driven extensions provide significant gains.

(b) Figure 12 shows the results for 200K data samples in scenarios where a positive expected gain exists, but the best conditioning attribute set is predicted to coincide with the preference attribute set itself. As expected, in the synthetic case, all strategies perform similarly with the preference-attribute based de-correlation or $ddSky$. In the real data case, we see that $lnSky$ has a somewhat lower reduction in execution time, indicating that the causal gain prediction does not perfectly identify these scenarios.

(c) Figure 13 shows the results for 200K data samples in scenarios where no conditioning attribute set with positive expected gain exists⁷. As expected, conditioning on an attribute set with negative expected gain significantly hurts performance. Interestingly, $gnSky$ shows improvements in certain cases with real causal graphs. This likely occurs because $gnSky$ misclassifies conditioning attribute sets: it sometimes assigns negative gain to sets that actually have positive gain. Conversely, this also explains why $gnSky$ performs worse than $lnSky$ in cases with true positive gain, as it incorrectly predicts negative conditioning attribute sets to yield positive gain. In contrast, if $lnSky$ predicts that a conditioning attribute set has negative gain, it should be avoided. In such cases, the non-causal version is preferable.

7.2.2 Impact of the Number of Tuples. As we see in Figure 14, for both synthetic and real-world graphs, performance gains improve as the size of the data gets larger, indicating that the proposed $lnSky$ de-correlation strategy enables the skyline algorithms to scale better with the data size.

⁷Note that for real causal graphs, we did not observe any scenario where $ddSky$ finds a conditioning attribute set with negative gain.

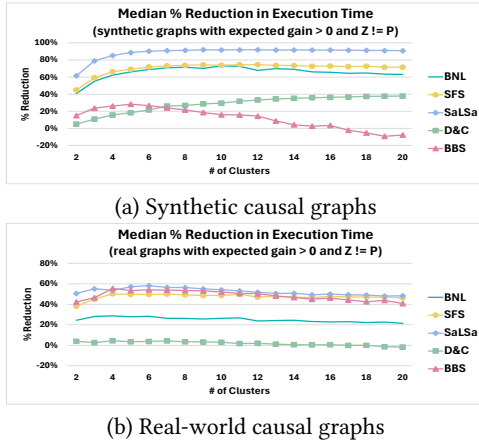


Figure 16: Median cost reduction (the higher, the better) for different numbers of de-correlation clusters when using $lnSky(0.6, 0.4)$; for these experiments, the expected gain is positive and larger than the gain predicted for the preference attribute set (i.e., we expect to see benefits over both vanilla skylines and preference attribute de-correlation) – 200K data

7.2.3 Effect of the $lnSky$ Parameters $\langle \lambda_o, \lambda_b \rangle$. Figure 15 shows the effect of leakage parameters on $lnSky$ search time. For synthetic graphs, $lnSky(0.6, 0.4)$ performs the best. For real-world graphs, the best option is $lnSky(0.8, 0.2)$. In the rest of this section, we use the leakage parameter pair $\langle 0.6, 0.4 \rangle$ for both synthetic and real-world graphs for overall experimental setup consistency, even though that pair is not the best option for real-world graphs.

7.2.4 Effect of Number of De-correlation Clusters. As outlined in Section 4, the proposed CSS approach relies on K-means clusters of (suitably normalized) data to achieve selective de-correlation. Figure 16 illustrates how the number of de-correlation clusters affects the skyline search efficiency. As we see in the figure, a small number of clusters (on the order of 10-15 clusters for 200K data) is sufficient to obtain the necessary de-correlation – a larger number of clusters either do not provide additional benefits or (in the case of causal BNL) actually starts hurting the performance due to the increasing merge overhead.

7.2.5 Impact of Inferred Causal Graphs. In the preceding experiments, we assumed that both the causal graph structure and the corresponding edge strengths were user provided. In practice, however, such causal knowledge may not be available. In such scenarios causal inference and causal effect estimation techniques can help infer the underlying causal graph and causal strengths. In this section, instead of using the user provided graphs, we use the causal graphs inferred by the NOTEARS [64] causal inference algorithm from (a) synthetic data (i.e., data generated from synthetic causal graphs) and (b) real-world data. Figure 17 compares the performance of $lnSky(0.6, 0.4)$ when using causal graphs inferred by the NOTEARS algorithm with its performance when using the user provided causal graphs, as well as with the baseline variants of the skyline algorithms. As we see in the figure, whether the causal graphs are user provided or inferred, $lnSky$ provides significant gains. For data obtained through synthetic causal graphs, using

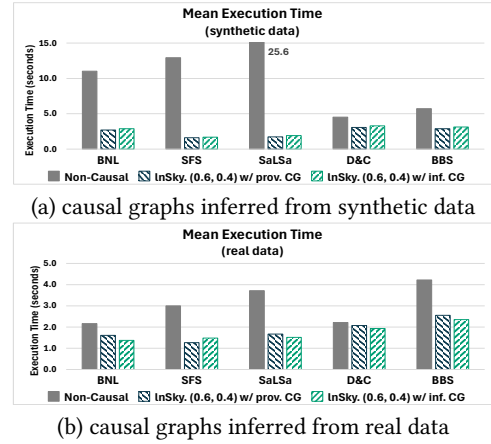


Figure 17: Cost of skyline computation with causal graphs inferred from (a) synthetic data (i.e., data generated from synthetic causal graphs) and (b) real data (the lower, the better), when using $lnSky(0.6, 0.4)$ – 200K data

inferred graphs performs slightly worse than using the true causal graphs. Interestingly, for real causal graphs, $lnSky$ using the inferred causal graphs outperforms $lnSky$ using domain-expert graphs, further demonstrating that a perfect causal graph is not required for CSS to provide significant benefits.

8 Conclusions

In this paper, we introduced Causal Search for Skylines (CSS), a novel approach that leverages pruning opportunities arising from positively aligned data distributions in skyline queries. CSS exploits the causal structure of the data to selectively de-correlate attributes, preserving positive correlations while removing negative ones. Our experiments showed that CSS achieves greater efficiency than existing methods, and we discussed its potential limitations. We also outlined a strategy for applying CSS in cases where it offers clear benefits while avoiding it otherwise.

One line of work we will explore in the future is to see how our divide-and-conquer style approach can be further improved by methods, such as parallelizing the skyline computation of individual clusters. Also, in CSS, we use the same skyline algorithm for computing both the skyline of each cluster and also to compute the final skyline after combining the skyline output of all the clusters; however, the skyline merge step can potentially be implemented more efficiently. In addition to these, we plan to explore other ways of "selective de-correlation", such as arranging the order of the variates in a causally informed manner and, generating new attributes through combinations of the preference attributes to produce causally derived conditioning sets.

Acknowledgements

This work is supported by NSF grant 2311716, "CausalBench: A Cyberinfrastructure for Causal-Learning Benchmarking for Efficiency, Reproducibility, and Scientific Collaboration", along with NSF grants 2309030, 2412115, 2435886 and a USACE grant GR40695.

GitHub Co-Pilot was used to assist in the code development. ChatGPT was used for minor editorial support to improve spelling, grammar, punctuation, and clarity, as specified in ACM guidelines [66].

References

- [1] S. Börzsönyi, D. Kossmann, and K. Stocker, "The skyline operator," in *ICDE*, 2001, pp. 421–430.
- [2] R. E. Steuer, *Multiple Criteria Optimization: Theory, Computation and Application*. John Wiley, New York, 546 pp, 1986.
- [3] J. Chomicki, P. Godfrey, J. Gryz, and D. Liang, "Skyline with presorting," in *ICDE*, 2003, pp. 717–719.
- [4] K.-L. Tan, P.-K. Eng, and B. C. Ooi, "Efficient progressive Skyline computation," in *VLDB*, 2001, pp. 301–310.
- [5] D. M. Chickering, "Optimal structure identification with greedy search," *Journal of machine learning research*, vol. 3, no. Nov, pp. 507–554, 2002.
- [6] P. Spirtes, C. N. Glymour, and R. Scheines, *Causation, prediction, and search*. MIT press, 2000.
- [7] S. Shimizu, P. O. Hoyer, A. Hyvärinen, A. Kerminen, and M. Jordan, "A linear non-gaussian acyclic model for causal discovery," *Journal of Machine Learning Research*, vol. 7, no. 10, 2006.
- [8] C. Chan, H. V. Jagadish, K. Tan, A. K. H. Tung, and Z. Zhang, "Finding k-dominant skylines in high dimensional space," in *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, 2006, pp. 503–514.
- [9] X. Zhang and H. Cavusoglu, "Skyline operators in multi-criteria decision making: A review of characterization, comparison, and perspectives," *Expert Systems with Applications*, vol. 306, p. 130889, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S095741742504504X>
- [10] G. S. Schlake and C. Beecks, "The skyline operator to find the needle in the haystack for automated clustering," in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 6117–6122.
- [11] R. Amin, T. Djatna, A. Annisa, and S. Sitanggang, "Development of skyline query algorithm for individual preference recommendation in streaming data," *Journal of Applied Data Sciences*, vol. 6, no. 2, pp. 1012–1025, 2025.
- [12] W. T. Hsu, Y. T. Wen, L. Y. Wei, and W. C. Peng, "Skyline travel routes: Exploring skyline for trip planning," in *2014 IEEE 15th International Conference on Mobile Data Management*, vol. 2, 2014, pp. 31–36.
- [13] Y.-T. Wen, K.-J. Cho, W.-C. Peng, J. Yeo, and S.-w. Hwang, "Kstr: Keyword-aware skyline travel route recommendation," in *2015 IEEE International Conference on Data Mining*, 2015, pp. 449–458.
- [14] H.-T. Kung, F. Luccio, and F. P. Preparata, "On finding the maxima of a set of vectors," *J. ACM*, vol. 22, pp. 469–476, 1975.
- [15] A. Vlachou, C. Doulkeridis, and N. Polyzotis, "Skyline query processing over joins," in *SIGMOD*, 2011.
- [16] J. Matoušek, "Computing dominances in E^n ," *Information Processing Letters*, vol. 38, pp. 277–278, 1991.
- [17] I. Stojmenović and M. Miyakawa, "An optimal parallel algorithm for solving the maximal elements problem in the plane," *Parallel Computing*, vol. 7, pp. 249–251, 1988.
- [18] P. Wu, C. Zhang, Y. Feng, B. Y. Zhao, D. Agrawal, and A. E. Abbadi, "Parallelizing Skyline queries for scalable distribution," in *EDBT*, 2006, pp. 112–130.
- [19] A. Vlachou, C. Doulkeridis, and Y. Kotidis, "Angle-based space partitioning for efficient parallel Skyline computation," in *SIGMOD Conference*, 2008, pp. 227–238.
- [20] J. Huang, J. Chen, Q. Du, and J. Yin, "A load balancing skyline query algorithm in high bandwidth distributed systems," in *FSKD*, vol. 5, 2010, pp. 2076–2080.
- [21] D. Kossmann, F. Ramsak, and S. Rost, "Shooting stars in the sky: An online algorithm for Skyline queries," in *VLDB*, 2002.
- [22] D. Papadias, Y. Tao, G. Fu, and B. Seeger, "Progressive Skyline computation in database systems," *ACM Trans. Database Systems*, vol. 30, pp. 41–82, 2005.
- [23] K. C. K. Lee, B. Zheng, H. Li, and W.-C. Lee, "Approaching the skyline in Z order," in *VLDB*, 2007, pp. 279–290.
- [24] D. Sun, S. Wu, J. Li, and A. K. H. Tung, "Skyline-join in distributed databases," in *ICDE Workshops*, 2008.
- [25] M. Nagendra and K. S. Candan, "Skyline-sensitive joins with LR-pruning," in *EDBT*, 2012, pp. 252–263.
- [26] X. Lin, Y. Yuan, W. Wang, and H. Lu, "Stabbing the sky: Efficient skyline computation over sliding windows," in *ICDE*, 2005.
- [27] Y. Tao and D. Papadias, "Maintaining sliding window skylines on data streams," *IEEE TKDE*, vol. 18, pp. 377–391, 2006.
- [28] W. Zhang, X. Lin, Y. Zhang, W. Wang, and J. X. Yu, "Probabilistic skyline operator over sliding windows," in *ICDE*, 2009, pp. 1060–1071.
- [29] I. Bartolini, P. Ciaccia, and M. Patella, "Salsa: computing the skyline without scanning the whole sky," in *CIKM*, 2006.
- [30] J. L. Bentley, K. L. Clarkson, and D. B. Levine, "Fast linear expected-time algorithms for computing maxima and convex hulls," *Algorithmica*, vol. 9, pp. 168–183, 1993.
- [31] P. Godfrey, R. Shipley, J. Gryz *et al.*, "Maximal vector computation in large data sets," in *VLDB*, vol. 5, 2005, pp. 229–240.
- [32] S. Park, T. Kim, J. Park, J. Kim, and H. Im, "Parallel skyline computation on multicore architectures," in *Proceedings of the 25th IEEE International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2009, pp. 760–771.
- [33] K. C. Lee, B. Zheng, H. Li, and W.-C. Lee, "Approaching the skyline in z order," in *VLDB*, vol. 7, 2007, pp. 279–290.
- [34] H. Shang and M. Kitsuregawa, "Skyline operator on anti-correlated distributions," *Proc. VLDB Endow.*, vol. 6, no. 9, pp. 649–660, 2013. [Online]. Available: <http://www.vldb.org/pvldb/vol6/p649-shang.pdf>
- [35] O. Barndorff-Nielsen and M. Sobel, "On the distribution of the number of admissible points in a vector random sample," *Theory of Probability & Its Applications*, vol. 11, no. 2, pp. 249–269, 1966.
- [36] J. L. Bentley, H.-T. Kung, M. Schkolnick, and C. D. Thompson, "On the average number of maxima in a set of vectors and applications," *Journal of the ACM (JACM)*, vol. 25, no. 4, pp. 536–543, 1978.
- [37] S. Chaudhuri, N. Dalvi, and R. Kaushik, "Robust cardinality and cost estimation for Skyline operator," in *ICDE*, 2006, p. 64.
- [38] Z. Zhang, Y. Yang, R. Cai, D. Papadias, and A. Tung, "Kernel-based skyline cardinality estimation," in *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 509–522. [Online]. Available: <https://doi-org.ezproxy1.lib.asu.edu/10.1145/1559845.1559899>
- [39] C. Luo, Z. Jiang, W.-C. Hou, S. He, and Q. Zhu, "A sampling approach for skyline query cardinality estimation," *Knowledge and information systems*, vol. 32, no. 2, pp. 281–301, 2012.
- [40] X. Miao, Y. Wu, J. Peng, Y. Gao, and J. Yin, "Efficient and effective cardinality estimation for skyline family," *Proc. ACM Manag. Data*, vol. 1, no. 1, May 2023. [Online]. Available: <https://doi-org.ezproxy1.lib.asu.edu/10.1145/3588958>
- [41] J. Peters, D. Janzing, and B. Schölkopf, *Elements of Causal Inference: Foundations and Learning Algorithms*, ser. Adaptive Computation and Machine Learning. Cambridge, MA: MIT Press, 2017.
- [42] B. Gao and Y. Cui, "Learning directed acyclic graphical structures with genetical genomics data," *Bioinformatics*, vol. 31, no. 24, pp. 3953–3960, 09 2015.
- [43] F. T. Azad, K. S. Candan, A. Kapkic, M. Li, H. Liu, P. Mandal, P. Sheth, B. Arslan, G. Chowell-Puente, J. Sabo, R. Muenich, J. R. Anton, and M. L. Sapino, "(vision paper) A vision for spatio-causal situation awareness, forecasting, and planning," *ACM Trans. Spatial Algorithms Syst.*, vol. 10, no. 2, p. 14, 2024. [Online]. Available: <https://doi.org/10.1145/3672556>
- [44] D. Janzing, "Causal regularization," in *Advances in Neural Information Processing Systems*, vol. 32. Vancouver, BC: Curran Associates, Inc., 2019.
- [45] D. Lopez Paz, R. Nishihara, S. Chintala, B. Scholkopf, and L. Bottou, "Discovering causal signals in images," in *CVPR*, July 2017.
- [46] E. Bareinboim, A. Forney, and J. Pearl, "Bandits with unobserved confounders: A causal approach," in *NIPS*, vol. 28, 2015.
- [47] M.-L. Li, K. S. Candan, and M. L. Sapino, "Causally Informed Factorization Machines," in *2024 IEEE International Conference on Big Data (BigData)*. Los Alamitos, CA, USA: IEEE Computer Society, Dec. 2024, pp. 448–455. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/BigData62323.2024.10825754>
- [48] M. Li, K. S. Candan, and M. L. Sapino, "CTT: causally informed tensor train decomposition," in *IEEE International Conference on Big Data, BigData 2023, Sorrento, Italy, December 15-18, 2023*. IEEE, 2023, pp. 1180–1187. [Online]. Available: <https://doi.org/10.1109/BigData59044.2023.10386626>
- [49] P. Sheth, R. Guo, L. Cheng, H. Liu, and K. S. Candan, "Causal disentanglement for implicit recommendations with network information," *ACM Trans. Knowl. Discov. Data*, vol. 17, no. 7, pp. 94:1–94:18, 2023. [Online]. Available: <https://doi.org/10.1145/3582435>
- [50] J. Pearl, "Causal inference in statistics: An overview," *Statist. Surv.*, vol. 3, pp. 96–146, 2009.
- [51] S. Wright, "Correlation and causation," *Journal of Agricultural Research*, vol. 20, pp. 557–585, 1921.
- [52] J. Pearl, *Causality: Models, Reasoning, and Inference*, 1st ed. Cambridge University Press, 2000.
- [53] W. Nash, T. Sellers, S. Talbot, A. Cawthorn, and W. Ford, "Abalone [dataset]," UCI Machine Learning Repository, 1994. [Online]. Available: <https://doi.org/10.24432/C55C7W>
- [54] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, DOI: <https://doi.org/10.24432/C5XW20>.
- [55] R. Quinlan, "Auto MPG," UCI Machine Learning Repository, 1993, DOI: <https://doi.org/10.24432/C5859H>.
- [56] L.-C. Yeh, "Concrete Compressive Strength," UCI Machine Learning Repository, 1998, DOI: <https://doi.org/10.24432/C5PK67>.
- [57] "Dry Bean," UCI Machine Learning Repository, 2020, DOI: <https://doi.org/10.24432/C50S4B>.
- [58] Hindy, "Hong kong weather (1970-2019)," Jan 2021. [Online]. Available: <https://www.kaggle.com/datasets/hindy51/hong-kong-weather-20082016/data>

- [59] G. Hebrail and A. Berard, "Individual Household Electric Power Consumption," UCI Machine Learning Repository, 2006, DOI: <https://doi.org/10.24432/C58K54>.
- [60] "Seoul Bike Sharing Demand," UCI Machine Learning Repository, 2020, DOI: <https://doi.org/10.24432/C5F62R>.
- [61] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Wine Quality," UCI Machine Learning Repository, 2009, DOI: <https://doi.org/10.24432/C56S3T>.
- [62] J. Beinecke and D. Heider, "Gaussian noise up-sampling is better suited than smote and adasyn for clinical decision making," *BioData Mining*, vol. 14, no. 1, p. 49, 2021.
- [63] P. Branco, L. Torgo, and R. P. Ribeiro, "A survey of predictive modeling on imbalanced domains," *ACM Comput. Surv.*, vol. 49, no. 2, Aug. 2016. [Online]. Available: <https://doi.org/10.1145/2907070>
- [64] X. Zheng, B. Aragam, P. K. Ravikumar, and E. P. Xing, "Dags with no tears: Continuous optimization for structure learning," *Advances in neural information processing systems*, vol. 31, 2018.
- [65] K. Keahey, J. Anderson, Z. Zhen, P. Riteau, P. Ruth, D. Stanzione, M. Cevik, J. Collieran, H. S. Gunawi, C. Hammock, J. Mambretti, A. Barnes, F. Halbach, A. Rocha, and J. Stubbs, "Lessons learned from the chameleon testbed," in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.
- [66] Acm policy on authorship. Association for Computing Machinery. [Online]. Available: <https://www.acm.org/publications/policies/new-acm-policy-on-authorship>

Table 3: Summary of the key symbols and notations

Symbol	Description
D	Input dataset consisting of tuples/points
A	Set of all attributes (feature space)
$P \subseteq A$	Set of preference (skyline) attributes
t	A tuple (data point) in dataset D
Θ	Set of preference criteria for attributes in P
θ_a	Preference criterion for attribute a (e.g., min or max)
S	Skyline set (non-dominated tuples)
$R(A)$	Relation defined over attribute set A
G or G_A	Causal graph over attributes in A
(V_A, E_A)	Vertices and edges of the causal graph
$\lambda(e)$	Sign/label of a causal edge e (+ or -)
C_P	Correlation matrix for preference attributes P
C_P^*	Correlation matrix for preference attributes P after conditioning on attributes in $\mathcal{Z}$
$C_P[i, j]$ and $C_P^*[i, j]$	Correlation between preference attributes a_i and a_j
$\mathcal{Z}$	Conditioning (grouping) attribute set
$\mathcal{Z}^-$	Selected conditioning set for de-correlation
$D_k(\mathcal{Z})$	Subset of data conditioned on $\mathcal{Z}$
$Paths_{i,j}$	All causal paths between a_i and a_j
$Open_{i,j}^+$	Open causal paths with positive overall effect
$Open_{i,j}^-$	Open causal paths with negative overall effect
$gain(\mathcal{Z})$	Gain from conditioning on $\mathcal{Z}$ (gnSky)
$l_gain(\mathcal{Z})$	Leaky gain from conditioning on $\mathcal{Z}$ (lnSky)
$Prov_{i,j}^+(\mathcal{Z})$	Provisionally opened positive paths due to conditioning
$Prov_{i,j}^-(\mathcal{Z})$	Provisionally opened negative paths due to conditioning
$Blocked_{i,j}^+(\mathcal{Z})$	Positive paths blocked by conditioning
$Blocked_{i,j}^-(\mathcal{Z})$	Negative paths blocked by conditioning
$imp_{i,j}^+(\mathcal{Z})$	Impact of conditioning on positive paths (gnSky)
$imp_{i,j}^-(\mathcal{Z})$	Impact of conditioning on negative paths (gnSky)
$l_imp_{i,j}^+(\mathcal{Z})$	Leaky impact on positive paths (lnSky)
$l_imp_{i,j}^-(\mathcal{Z})$	Leaky impact on negative paths (lnSky)
$cw(p)$	Causal weight of path p
λ_o, λ_b	Information-passing factors for open and blocked nodes

A Appendix - A: Preliminaries

Table 3 provides the key notations used in the paper.

A.1 Tuple Dominance and Skylines

Let D be a set of data points in a feature/attribute space, A , and the set, $P = a_1, \dots, a_p \subseteq A$, denote the set of preference attributes. Furthermore, let $\Theta = \{\theta_{a_1}, \dots, \theta_{a_p}\}$, denote the corresponding preference criteria, such that $\theta_* \in \{min, max\}$. According to the conventional definition [1] of dominance, a tuple t *dominates* another tuple q if and only if tuple t is better than or equal to ($\geq$) tuple q in all preference attributes and better than ($>$) q in at least one attribute of the preference attribute set.

DEFINITION 3 (TUPLE DOMINANCE (*dom*)). Let u and t be two tuples in D and $t.a_h$ be the value of attribute, a_h , of tuple t . Let P be the user provided preference attribute set. Tuple u dominates t ($u \text{ dom}_P t$) in the preference attribute set P iff

$$(\forall_{a_i \in P} u.a_i \geq_{a_i} t.a_i) \wedge (\exists_{a_k \in P} u.a_k >_{a_k} t.a_k),$$

where $>_{a_*}$ and $\geq_{a_*}$ are the value dominance relationships according to the preference function θ_{a_*} . $\diamond$

Given this, the skyline of the dataset, D , then consists of the subset of tuples that are not dominated by any other tuple in D :

DEFINITION 4 (SKYLINE). Let D be a data set. The skyline, $S \subseteq D$, with respect to the preference attribute set P and preference criteria Θ is the maximal subset of D , where

$$S = \{t \in D \mid \nexists_{u \in D} u \text{ dom}_P t\}.$$

$\diamond$

Note that, from these, it is possible to infer that

$$(\forall_{u \in D \setminus S} \exists_{t \in S} t \text{ dom}_P u) \wedge (\nexists_{u, t \in S} t \text{ dom}_P u).$$

A.2 Causal Graphs

Let D be a data set defined in an attribute space, A . A causal graph, $G_A = (V_A, E_A, \lambda)$, corresponding to the attribute set A is an edge-labeled directed acyclic graph (DAG), where for each attribute $a_i \in A$, there is a vertex $v_i \in V_A$ and each edge $e_h = \langle a_i, a_j \rangle$ (or $a_i \rightarrow a_j$) indicates a known direct causal relationship between the attributes a_i and a_j and the edge label $\lambda(e_h)$ would indicate the nature of the causal relationship between the two attributes.

In a simple causal model, the label $\lambda(e_h) \in \{+, -\}$ would indicate positive or negative causal impact. Intuitively, this indicates that in the context in which the data set D has been collected, there is a process that governs the relationship between the values of a_i and a_j ; in other words, the values of a_i and a_j cannot be independently varied – any change in the value of a_i comes with a corresponding change in the value of a_j as described by the edge label (but not vice versa).

EXAMPLE 5 (CAUSAL GRAPH FOR HOUSE HUNTING). Let us consider our running example – house hunting. Figure 1(b) shows a causal graph, G , that outlines the causal relationships among the attributes of the data set. In this example, we see that the Distance_to_city_center attribute causally impacts both Price and Commute attributes of the data. $\circ$

In linear structural causal models [51, 52], the label $\lambda(e_h)$ associated with a causal edge $e_h = a_i \rightarrow a_j$ would be of the form $a_j = c_{ij}a_i + \epsilon_{ij}$, where the constant c_{ij} denotes the causal impact of a_i on a_j , whereas ϵ_{ij} would denote the random noise that acts on attribute a_j along this causal edge.

A.3 Causal Paths

A causal path, then, is a sequence of (backward or forward) causal edges that causally relates two given attributes; moreover, the causal path is said to be a directed causal path if all of the edges in the sequence point in the cause-effect direction:

DEFINITION 5 (CAUSAL PATH). Let $G_A = (V_A, E_A)$ be a causal graph on attribute set A and let a_i and a_j be two attributes in A . The sequence $[e_1, e_2, \dots, e_k]$ of edges is said to be a causal path from a_i to a_j (denoted as $a_i \rightsquigarrow a_j$) iff there is a sequence $[a_{s_1}, \dots, a_{s_k}, a_{s_{k+1}}]$ of attributes, such that

$$(\forall_{1 \leq h \leq k} (e_h = \langle a_{s_h}, a_{s_{h+1}} \rangle) \vee (e_h = \langle a_{s_{h+1}}, a_{s_h} \rangle)) \wedge (a_{s_1} = a_i) \wedge (a_{s_{k+1}} = a_j).$$

A causal path is said to be a directed causal path (denoted as $a_i \rightarrow \dots \rightarrow a_j$) iff

$$(\forall_{1 \leq h \leq k} e_h = \langle a_{s_h}, a_{s_{h+1}} \rangle) \wedge (e_{s_1} = a_i) \wedge (e_{s_{k+1}} = a_j).$$

All causal paths that are not directed are referred to as undirected causal paths. $\diamond$

EXAMPLE 6 (SAMPLE PATHS ON THE HOUSE HUNTING CAUSAL GRAPH). In our running example in Figure 1, the edge sequence $[e_4, e_5]$ is a directed path from the Distance_to_city_center attribute to the Price attribute. The edge sequence $[e_1, e_2]$, on the other hand, is an undirected causal path from the Commute attribute to the Price attribute. $\circ$

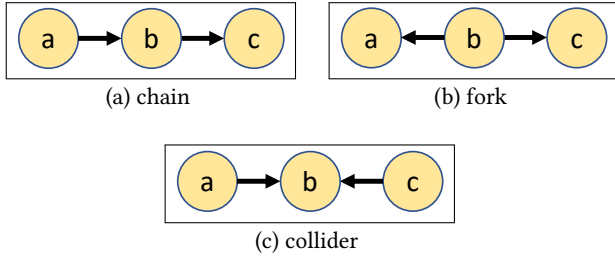


Figure 18: Three basic causal structures

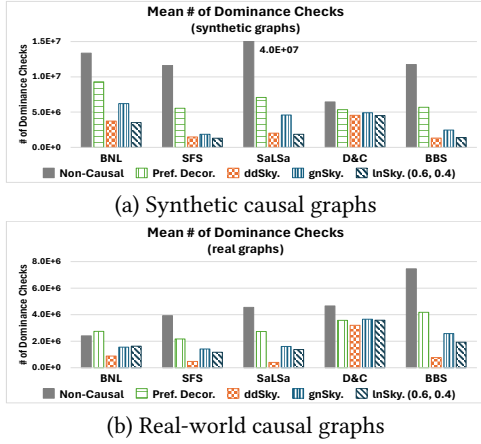


Figure 19: Cost of skyline computation with (a) synthetic and (b) real-world causal graphs (the lower, the better) – 200K data

A.4 Causal Graph Patterns and Statistical Relationships

Figure 18 presents three basic causal structures introduced in [50] - (a): a *chain* structure where a causally affects c through its influence on b (here b is referred to as a mediator); (b): a *fork* structure where b is the common cause of both a and c – note that, in this case, a and c are likely dependent but there is no causation between them; (c): a *collider* structure where both a and c independently cause b , but there is no causation between them.

B Appendix - B: Dominance Check Results

In Section 7 of the paper, we reported the execution times of the skyline algorithms and the execution time gains of CSS. In this section, in Figures 19 through 26, we report the corresponding dominance check results, which parallel the performance results reported earlier in Figures 9 through 17.

C Appendix - C: Computing Conditioned Gain Values

As outlined in Section 6.5, for data optimized selective de-correlation for skyline search, we need a mechanism to quantify the degrees of leakage and noise as a function of the structure of the causal graph. More specifically, let p be a path between two

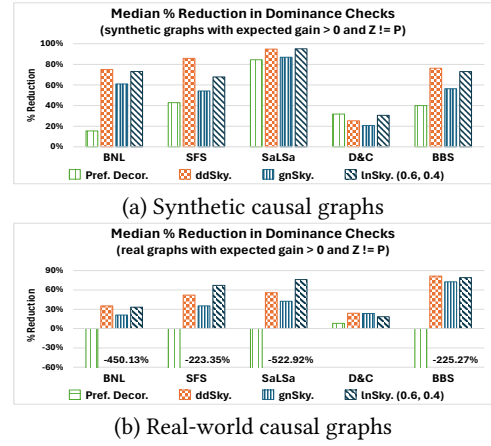


Figure 20: Median cost reduction (the higher, the better); for these experiments, the expected gain is positive and larger than the gain predicted for the preference attribute set (i.e., we expect to see benefits over both vanilla skylines and preference attribute de-correlation) – 200K data

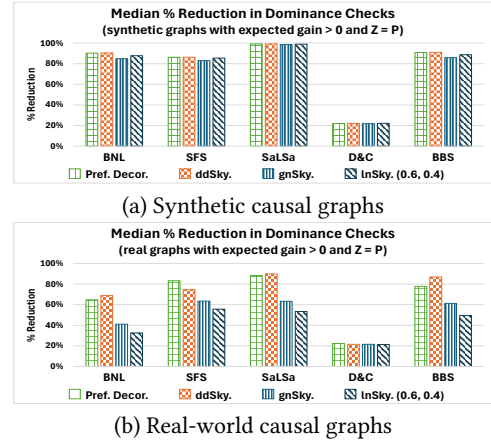


Figure 21: Median cost reduction (the higher, the better); for these experiments, the expected gain is positive and equal to the gain predicted for the preference attribute set (i.e., we do not expect to beat preference attribute de-correlation) – 200K data

preference variables X and Y and $\mathcal{Z}$ be the set of conditioning attributes. We compute the conditioning gain for the path given the conditioning set $\mathcal{Z}$ as

$$c_gain(p, \mathcal{Z}) = corr_{p, \mathcal{Z}}(X, Y) - corr_p(X, Y),$$

where

- $corr_p(X, Y)$ is the correlation between X and Y , implied by the causal path p , before the conditioning step, and
- $corr_{p, \mathcal{Z}}(X, Y)$; i.e., the correlation between X and Y , implied by the causal path p , after the conditioning of attributes in $\mathcal{Z}$.

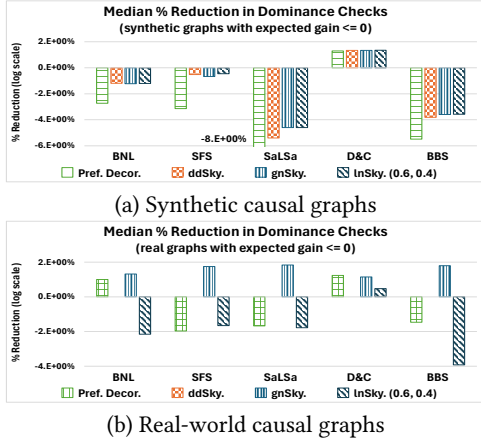


Figure 22: Median cost reduction (the higher, the better); for these experiments, the expected gain is negative (i.e., we do not expect to see benefits from de-correlation) – 200K data

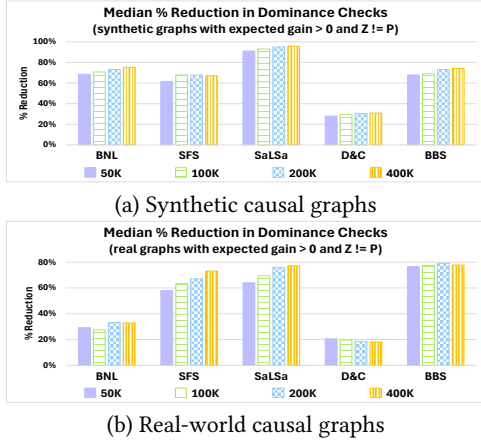


Figure 23: Median cost reduction (the higher, the better) for different numbers of tuples when using $lnSky(0.6, 0.4)$; for these experiments, the expected gain is positive and larger than the gain predicted for the preference attribute set

Given these, the conditioning gain for the path given the conditioning set $\mathcal{Z}$ can be computed as

$$c_gain(p, \mathcal{Z}) = corr_{p, \mathcal{Z}}(X, Y) - corr_p(X, Y).$$

In this section, we discuss how to compute these terms. In particular, we will make the simplifying assumption that the individual variables have normal distributions and sets of variables have multi-variate Gaussian distributions.

C.1 Computing the Unconditioned Correlation

Let us first consider computation of the unconditioned correlation, $corr_p(X, Y)$ between attributes X and Y on the path p (we will drop the subscript p , when it can be treated as implied). Let $X = (U_0)$ and $Y = (U_m)$ be two Gaussian variables that are related to each other through a sequence of intermediary Gaussian variables,

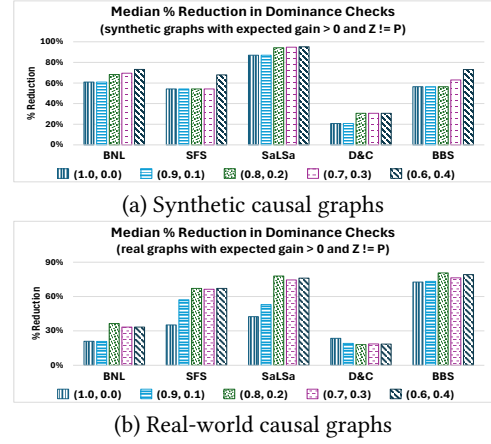


Figure 24: Median cost reduction (the higher, the better) for different $\langle \lambda_o, \lambda_b \rangle$ pairs; for these experiments, the expected gain is positive and larger than the gain predicted for the preference attribute set – 200K data

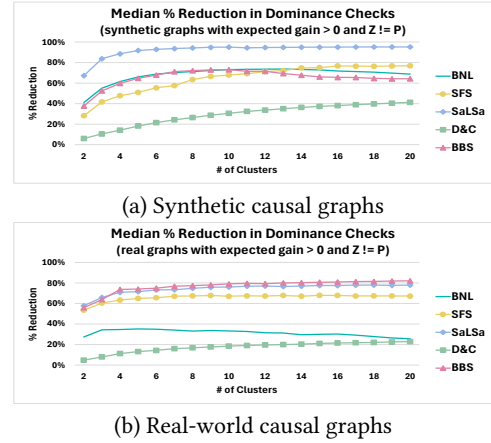


Figure 25: Median cost reduction (the higher, the better) for different numbers of de-correlation clusters when using $lnSky(0.6, 0.4)$; for these experiments, the expected gain is positive and larger than the gain predicted for the preference attribute set – 200K data

$U_1, \dots, U_{m-1}$, that define the path,

$$\Pi = X \xrightarrow{\{f,b\}} U_1 \xrightarrow{\{f,b\}} U_2 \xrightarrow{\{f,b\}} \dots \xrightarrow{\{f,b\}} U_{m-1} \xrightarrow{\{f,b\}} Y,$$

where each arrow is pointing either forward or backward. Let each forward edge $U_i \rightarrow U_{i+1}$ represent a linear relationship where the amount $\alpha_{i,i+1}U_i + \epsilon_{i,i+1}$, is contributed to U_{i+1} by U_i ; here U_i, U_{i+1} , and $\epsilon_{i,i+1}$ are Gaussians. Similarly each backward edge $U_{i-1} \leftarrow U_i$ represents a linear relationship where the amount $U_i = \alpha_{i,i-1}U_{i-1} + \epsilon_{i,i-1}$ is contributed to U_{i-1} by U_i ; here, U_i, U_{i-1} , and $\epsilon_{i,i-1}$ are Gaussians.

Given the above, in order to leverage the formula

$$corr(X, Y) = \frac{cov(X, Y)}{\sqrt{var(X)} \cdot \sqrt{var(Y)}}$$

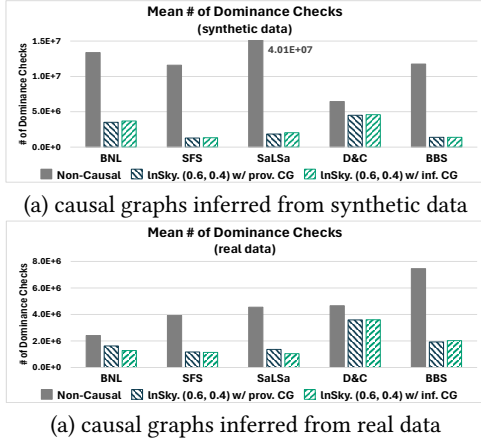


Figure 26: Cost of skyline computation with causal graphs inferred from (a) synthetic data (i.e., data generated from synthetic causal graphs) and (b) real data (the lower, the better), when using *InSky*(0.6, 0.4) – 200K data

for the computation of $\text{corr}(X, Y)$, we need to compute the terms $\text{var}(X)$, $\text{var}(Y)$, and $\text{cov}(X, Y)$.

C.1.1 Computing Covariance, $\text{cov}(X, Y)$. In order to compute $\text{cov}(X, Y)$, we need to consider three scenarios:

Path without Forks or Colliders. If the path does not contain any forks or colliders, we have a chain of mediators between X and Y . If X is the cause, then

$$\text{cov}(X, Y) = \left(\prod_{i=0}^{m-1} \alpha_{i,i+1} \right) \cdot \text{var}(X);$$

else if Y is the cause, then

$$\text{cov}(X, Y) = \left(\prod_{i=0}^{m-1} \alpha_{i+1,i} \right) \cdot \text{var}(Y).$$

Path without Colliders. If the path does not contain colliders, then the path can contain at most one fork. Let U_f be the fork of the form " $\leftarrow U_f \rightarrow$ " between X and Y . Given this, we have

$$\text{cov}(X, Y) = \left(\prod_{i=0}^{f-1} \alpha_{i,i+1} \right) \cdot \text{var}(U_f) \cdot \left(\prod_{i=f}^{m-1} \alpha_{i+1,i} \right).$$

Path with Colliders. If any of the intermediary variable is a collider of the form " $\rightarrow U_c \leftarrow$ ", then $\text{cov}(X, Y)$ implied by the path p is 0.

C.1.2 Computing Variance, $\text{var}(U_i)$. For $U_i \in \{X = U_0, \dots, U_{m-1}, Y = U_m\}$, we have the following situations:

- if U_i has no incoming edges on p , then $\text{var}(U_i)$ is the measured variance of the variable U_i .
- if U_i has incoming edges, then

$$\begin{aligned} \text{var}(U_i) &= [U_{i-1} \rightarrow U_i] \cdot (\alpha_{i-1,i}^2 \cdot \text{var}(U_{i-1}) + \epsilon_{i-1,i}) + \\ &\quad [U_i \leftarrow U_{i+1}] \cdot (\alpha_{i+1,i}^2 \cdot \text{var}(U_{i+1}) + \epsilon_{i+1,i}). \end{aligned}$$

Note that, this gives us a set of constraints from which we can obtain the variances for all variables on the path, including $\text{var}(X) = \text{var}(U_0)$ and $\text{var}(Y) = \text{var}(U_m)$.

C.2 Computing the Conditioned Correlation

In order to compute the conditioned correlation, $\text{corr}_{p,Z}(X, Y)$, we need to compute the conditioned variances and covariances on the path p . Let $Z = \{Z_1, \dots, Z_k\} \subseteq \{U_0, \dots, U_m\}$ be a set of conditioning variables and let $S = \{S_1, \dots, S_k\}$ be such that $\forall S_i, S_i \subseteq Z_i$ is a non-empty subset of Z_i , obtained through clustering, such that $S_i \sim \mathcal{N}(\mu_{S_i}, V_{S_i})$ preserves ρ_{U_i} of the overall variance; i.e., $\text{var}(U_i|S) = \rho_{U_i} \cdot \text{var}(Z_i)$ (see Section C.3). In order to compute $\text{corr}_{p,Z}(X, Y)$ using the equation

$$\text{corr}_{p,Z}(X, Y) = \text{corr}_p(X, Y|S) = \frac{\text{cov}_p(X, Y|S)}{\sqrt{\text{var}_p(X|S)} \cdot \sqrt{\text{var}_p(Y|S)}},$$

we need to compute the terms $\text{var}_p(X|S)$, $\text{var}_p(Y|S)$, and $\text{cov}_p(X, Y|S)$. Note that, as before, we will drop the subscript p , when it can be treated as implied.

C.2.1 Computing Conditional Variances, $\text{var}(X|S)$ and $\text{var}(Y|S)$. We first discuss how to compute $\text{var}(X|S)$ relying on the relationship between X and the conditioning variables on the path⁸. We need to consider three cases:

(Case 1). If X is causally independent of the conditioning variables on the path, then $\text{var}(X|S) = \text{var}(X)$.

(Case 2). Let U_1 be the conditioned variable closest to X on the path and let there be a linear relationship of the form $X = \alpha U_1 + \epsilon$ between X and U_1 . From the definition of conditional variance, we have

$$\text{var}(X) = E[\text{var}(X|U_1)] + \text{var}(E[X|U_1]),$$

which can be re-written as

$$\text{var}(X|S) = E[\text{var}(X|U_1, S)|S] + \text{var}(E[X|U_1, S]|S).$$

Since $(X \perp S|U_1)$, this can be simplified to

$$\text{var}(X|S) = \underbrace{E[\text{var}(X|U_1)|S]}_{\text{term1}} + \underbrace{\text{var}(E[X|U_1]|S)}_{\text{term2}}.$$

Again, since we have $(X \perp S|U_1)$, we can further simplify *term1* as

$$\begin{aligned} E[\text{var}(X|U_1)|S] &= \text{var}(X|U_1) \\ &= \text{var}(X) - \frac{\text{cov}(X, U_1)^2}{\text{var}(U_1)}. \end{aligned}$$

To compute *term2*, we can write the conditional expectation $E[X|U_1]|S$ as

$$\mu_{X|S} + \frac{\text{cov}(X, U_1|S)}{\text{var}(U_1|S)} \cdot (U_1|S - \mu_{U_1|S}).$$

where μ denotes the mean values. Taking the variance of this expectation gives us the following for *term2*:

$$\begin{aligned} \text{var}(E[X|U_1]|S) &= \frac{\text{cov}(X, U_1|S)^2}{\text{var}(U_1|S)^2} \cdot \text{var}(U_1|S) \\ &= \frac{\text{cov}(X, U_1|S)^2}{\text{var}(U_1|S)}. \end{aligned}$$

Combining *term1* and *term2*, we can compute $\text{var}(X|S)$ as

$$\text{var}(X|S) = \text{var}(X) - \frac{\text{cov}(X, U_1)^2}{\text{var}(U_1)} + \frac{\text{cov}(X, U_1|S)^2}{\text{var}(U_1|S)}.$$

⁸The term $\text{var}(Y|S)$ can be similarly computed.

As stated earlier, $\text{var}(U_1|S) = \rho_{U_1} \cdot \text{var}(Z_1)$, as discussed in Section C.3. We discuss the computation of the conditional covariances in Section C.2.2.

(Case 3). Let $\mathcal{U}_X \subseteq \{U_1, \dots, U_n\}$ be the set of conditioned variables closest to X with a linear relationship of the form $X = \alpha_1 U_1 + \dots + \alpha_n U_n + \epsilon$ among $X, U_1, \dots$, and U_n . Intuitively, the set $\mathcal{U}_X$ are the set of variables such that $X \perp S | \mathcal{U}_X$. From the definition of conditional variance, we have

$$\text{var}(X) = E[\text{var}(X|\mathcal{U}_X)] + \text{var}(E[X|\mathcal{U}_X]),$$

which can be re-written as

$$\text{var}(X|S) = E[\text{var}(X|\mathcal{U}_X, S)|S] + \text{var}(E[X|\mathcal{U}_X, S]|S).$$

Since $(X \perp S | \mathcal{U}_X)$, this can be simplified to

$$\text{var}(X|S) = \underbrace{E[\text{var}(X|\mathcal{U}_X)|S]}_{\text{term1}} + \underbrace{\text{var}(E[X|\mathcal{U}_X]|S)}_{\text{term2}}.$$

Again, since we have $(X \perp S | \mathcal{U}_X)$, we can further simplify *term1* as

$$\begin{aligned} E[\text{var}(X|\mathcal{U}_X)|S] &= \text{var}(X|\mathcal{U}_X) \\ &= \text{var}(X) - (\bar{c}_X \ C_{\mathcal{U}_X}^{-1} \ \bar{c}_X^T), \end{aligned}$$

where

- $\bar{c}_X = [\text{cov}(X, U_1) \dots \text{cov}(X, U_n)]$ and
- $C_{\mathcal{U}_X}$ is a covariance matrix, such that

$$C_{\mathcal{U}_X} = \begin{bmatrix} \text{var}(U_1) & \text{cov}(U_1, U_2) & \dots & \text{cov}(U_1, U_n) \\ \text{cov}(U_2, U_1) & \text{var}(U_2) & \dots & \text{cov}(U_2, U_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(U_n, U_1) & \text{cov}(U_n, U_2) & \dots & \text{var}(U_n) \end{bmatrix}.$$

In order to compute *term2*, we need to formulate $E[X|\mathcal{U}_X]|S$:

$$\mu_{X|S} + \left(\bar{c}_{X|S} \ C_{\mathcal{U}_X|S}^{-1} \ \left(\begin{bmatrix} U_1|S \\ \vdots \\ U_n|S \end{bmatrix} - \begin{bmatrix} \mu_{U_1|S} \\ \vdots \\ \mu_{U_n|S} \end{bmatrix} \right) \right),$$

where $\bar{c}_{X|S} = [\text{cov}(X, U_1|S) \dots \text{cov}(X, U_n|S)]$ and $C_{\mathcal{U}_X|S}$ is a covariance matrix, such that

$$C_{\mathcal{U}_X|S} = \begin{bmatrix} \text{var}(U_1|S) & \text{cov}(U_1, U_2|S) & \dots & \text{cov}(U_1, U_n|S) \\ \text{cov}(U_2, U_1|S) & \text{var}(U_2|S) & \dots & \text{cov}(U_2, U_n|S) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(U_n, U_1|S) & \text{cov}(U_n, U_2|S) & \dots & \text{var}(U_n|S) \end{bmatrix}.$$

Note that above $\forall_{i \neq j} \text{cov}(U_i, U_j|S) = 0$, due to clustering based conditioning which erases the covariance among the conditioned variables. Thus, we can simplify $E[X|\mathcal{U}_X]|S$ as

$$\mu_{X|S} + \frac{\text{cov}(X, U_1|S)}{\text{var}(U_1|S)} \cdot (U_1|S) + \dots + \frac{\text{cov}(X, U_n|S)}{\text{var}(U_n|S)} \cdot (U_n|S) + \text{constant}.$$

Given the above, *term2* (i.e., $\text{var}(E[X|\mathcal{U}_X]|S)$) can be computed as

$$\text{var}(E[X|\mathcal{U}_X]|S) = \frac{\text{cov}(X, U_1|S)^2}{\text{var}(U_1|S)} + \dots + \frac{\text{cov}(X, U_n|S)^2}{\text{var}(U_n|S)}.$$

Putting *term1* and *term2* together we obtain $\text{var}(X|S)$ as

$$\begin{aligned} \text{var}(X|S) &= \text{var}(X) - (\bar{c}_X \ C_{\mathcal{U}_X}^{-1} \ \bar{c}_X^T) + \\ &\quad \frac{\text{cov}(X, U_1|S)^2}{\text{var}(U_1|S)} + \dots + \frac{\text{cov}(X, U_n|S)^2}{\text{var}(U_n|S)}. \end{aligned}$$

We discuss the computation of the conditional covariances next.

C.2.2 Computing Conditional Covariance $\text{cov}(X, U_h|S)$. Let $U_h \in U_1, \dots, U_{m-1}, Y$; we first discuss how to compute $\text{cov}(X, U_h|S)$ relying on the relationship between X and the conditioning variables on the path⁹. We need to consider three cases:

(Case 1). If X is causally independent of U_h , ignoring the conditioning variables, then $\text{cov}(X, U_h|S) = 0$.

(Case 2). Let $U_h = U_1$, such that there is a linear relationship of the form $X = \alpha U_h + \epsilon$ between X and U_h . In this case, since $Z \perp \epsilon$, we have

$$\begin{aligned} \text{cov}(X, U_h|S) &= \text{cov}(X, U_1|S) \\ &= \text{cov}(X, U_1) \cdot \frac{\text{var}(U_1|S)}{\text{var}(U_1)}. \end{aligned}$$

(Case 3). Let $\mathcal{U}_X \subseteq \{U_1, \dots, U_n\}$ be the set of conditioned variables closest to X with a linear relationship of the form $X = \alpha_1 U_1 + \dots + \alpha_n U_n + \epsilon$ among $X, U_1, \dots$, and U_n . Intuitively, the set $\mathcal{U}_X$ are the set of variables such that $X \perp S | \mathcal{U}_X$. Similarly, let $\mathcal{U}_h$ be the set of variables such that $U_h \perp S | \mathcal{U}_h$. From the definition of conditional covariance, we have

$$\begin{aligned} \text{cov}(X, U_h) &= E[\text{cov}(X, U_h|\mathcal{U}_X, \mathcal{U}_h)] + \\ &\quad \text{cov}(E[X|\mathcal{U}_X, \mathcal{U}_h], E[U_h|\mathcal{U}_X, \mathcal{U}_h]). \end{aligned}$$

We can further write this as

$$\begin{aligned} \text{cov}(X, U_h|S) &= E[\text{cov}(X, U_h|\mathcal{U}_X, \mathcal{U}_h, S)|S] + \\ &\quad \text{cov}(E[X|\mathcal{U}_X, \mathcal{U}_h, S], E[U_h|\mathcal{U}_X, \mathcal{U}_h, S]|S). \end{aligned}$$

Since, $X, U_h \perp S | \mathcal{U}_X, \mathcal{U}_h$, we can rewrite the above equation as

$$\begin{aligned} \text{cov}(X, U_h|S) &= \underbrace{E[\text{cov}(X, Y|\mathcal{U}_X, \mathcal{U}_h)|S]}_{\text{term1}} + \\ &\quad \underbrace{\text{cov}(E[X|\mathcal{U}_X, \mathcal{U}_h], E[U_h|\mathcal{U}_X, \mathcal{U}_h]|S)}_{\text{term2}}. \end{aligned}$$

Again, since we have $X, U_h \perp S | \mathcal{U}_X, \mathcal{U}_h$, we can simplify *term1* as

$$\begin{aligned} E[\text{cov}(X, U_h|\mathcal{U}_X, \mathcal{U}_h)|S] &= \text{cov}(X, U_h|\mathcal{U}_X, \mathcal{U}_h) \\ &= \text{cov}(X, U_h) - (\bar{c}_X \ C_{\mathcal{U}_X}^{-1} \ \bar{c}_h^T), \end{aligned}$$

where, given the set of conditioning variables $\{U^1, \dots, U^r\} = \mathcal{U}_X \cup \mathcal{U}_h$, we have

- $\bar{c}_X = [\text{cov}(X, U^1) \dots \text{cov}(X, U^r)]$,
- $\bar{c}_h = [\text{cov}(U_h, U^1) \dots \text{cov}(U_h, U^r)]$, and
- $C_{\mathcal{U}_X, \mathcal{U}_h}$ is a covariance matrix, such that

$$C_{\mathcal{U}_X, \mathcal{U}_h} = \begin{bmatrix} \text{var}(U^1) & \text{cov}(U^1, U^2) & \dots & \text{cov}(U^1, U^r) \\ \text{cov}(U^2, U^1) & \text{var}(U^2) & \dots & \text{cov}(U^2, U^r) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(U^r, U^1) & \text{cov}(U^r, U^2) & \dots & \text{var}(U^r) \end{bmatrix}.$$

In order to compute *term2*, we first need to formulate $E[X|\mathcal{U}_X, \mathcal{U}_h]|S$ and $E[U_h|\mathcal{U}_X, \mathcal{U}_h]|S$. We can compute $E[X|\mathcal{U}_X, \mathcal{U}_h]|S$ as

$$\mu_{X|S} + \left(\bar{c}_{X|S} \ C_{\mathcal{U}_X, \mathcal{U}_h|S}^{-1} \ \left(\begin{bmatrix} U^1|S \\ \vdots \\ U^r|S \end{bmatrix} - \begin{bmatrix} \mu_{U^1|S} \\ \vdots \\ \mu_{U^r|S} \end{bmatrix} \right) \right),$$

⁹The term $\text{cov}(Y, U_h|S)$ can also be similarly computed.

where $\vec{c}_{X|S} = [\text{cov}(X, U^1|S) \dots \text{cov}(X, U^r|S)]$ and $\mathbf{C}_{\mathbf{U}_X, \mathbf{U}_h|S}$ is a covariance matrix, such that

$$\mathbf{C}_{\mathbf{U}_X, \mathbf{U}_h|S} = \begin{bmatrix} \text{var}(U^1|S) & \text{cov}(U^1, U^2|S) & \dots & \text{cov}(U^1, U^r|S) \\ \text{cov}(U^2, U^1|S) & \text{var}(U^2|S) & \dots & \text{cov}(U^2, U^r|S) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(U^r, U^1|S) & \text{cov}(U^r, U^2|S) & \dots & \text{var}(U^r|S) \end{bmatrix}.$$

Note that above $\text{cov}(U^i, U^j|S) = 0$, due to clustering based conditioning which erases the covariance among the conditioned variables. Thus, we can simplify $E[X|\mathbf{U}_X, \mathbf{U}_h|S]$ as

$$\mu_{X|S} + \frac{\text{cov}(X, U^1|S)}{\text{var}(U^1|S)} \cdot (U^1|S) + \dots + \frac{\text{cov}(X, U^r|S)}{\text{var}(U^r|S)} \cdot (U^r|S) + \text{constant}.$$

We can similarly write $E[U_h|\mathbf{U}_X, \mathbf{U}_h|S]$ as

$$\mu_{U_h|S} + \frac{\text{cov}(U_h, U^1|S)}{\text{var}(U^1|S)} \cdot (U^1|S) + \dots + \frac{\text{cov}(U_h, U^r|S)}{\text{var}(U^r|S)} \cdot (U^r|S) + \text{constant}.$$

Given the above, *term2* (i.e., $\text{cov}(E[X|\mathbf{U}_X, \mathbf{U}_h|S], E[U_h|\mathbf{U}_X, \mathbf{U}_h|S])$) can be computed as

$$= \frac{\text{cov}(X, U^1|S) \cdot \text{cov}(U_h, U^1|S)}{\text{var}(U^1|S)^2} \cdot \text{var}(U^1|S) + \dots + \frac{\text{cov}(X, U^r|S) \cdot \text{cov}(U_h, U^r|S)}{\text{var}(U^r|S)^2} \cdot \text{var}(U^r|S) + \text{cov_terms},$$

where *cov_terms* account for the conditional covariances among the variables in $\{U^1, \dots, U^r\} = \mathbf{U}_X \cup \mathbf{U}_h$.

Once again, since we have $\text{cov}(U^i, U^j|S) = 0$, for $i \neq j$, the above equation for *term2* simplifies to

$$= \frac{\text{cov}(X, U^1|S) \cdot \text{cov}(U_h, U^1|S)}{\text{var}(U^1|S)} + \dots + \frac{\text{cov}(X, U^r|S) \cdot \text{cov}(U_h, U^r|S)}{\text{var}(U^r|S)}$$

Note that the terms $\text{var}(U^1|S), \dots, \text{var}(U^r|S)$ can be computed as described in Section C.3. Thus, in order to compute $\text{cov}(X, U_h|S)$ by combining *term1* and *term2* obtained above, we only need to compute the terms $\text{cov}(X, U^1|S), \text{cov}(U_h, U^1|S), \dots, \text{cov}(X, U^r|S)$, and $\text{cov}(U_h, U^r|S)$ – note that the $\text{cov}(U_h, U^i|S)$ terms are equal to 0 unless $U^h = Y$.

C.3 Computing the Variance Scaling Factor, ρ

C.3.1 Variance Scaling for Truncated Gaussians. Let $Z \in \mathcal{Z}$ be a Gaussian random variable, $Z \sim \mathcal{N}(\mu_Z, V_Z)$, with mean, μ_Z and variance $\text{var}(Z) = V_Z$. Let S be a subset of Z with the values in the range of $[l, u]$. We can calculate the truncated variance $\text{var}(Z|S)$ as follows:

$$\begin{aligned} \text{var}(Z|S) &= \text{var}(Z | l \leq Z \leq u) \\ &= \text{var}(Z) \cdot \left(1 + \frac{U \cdot f(U) - L \cdot f(L)}{\Phi(L) - \Phi(U)} - \mu_{\text{cond}}^2 \right), \end{aligned}$$

where $f(x)$ is the PDF of the standard normal distribution (i.e., $\mathcal{N}(0, 1)$):

$$f(x) = \frac{1}{\sqrt{2\pi}} \cdot \exp\left(-\frac{x^2}{2}\right),$$

$\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution evaluated at x ,

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \cdot \int_{-\infty}^x \exp\left(-\frac{t^2}{2}\right) dt,$$

L and U are the boundaries scaled onto the standard normal distribution,

$$L = \frac{l - \mu_Z}{\sqrt{V_Z}} \quad \text{and} \quad U = \frac{u - \mu_Z}{\sqrt{V_Z}},$$

and, μ_{cond}^2 is the conditional mean of the variance, truncated at boundaries L and U :

$$\mu_{\text{cond}} = \frac{f(L) - f(U)}{\Phi(U) - \Phi(L)}.$$

In other words, given the truncation boundaries l and u of S , the scaling factor, ρ , for the variance of set S relative to the variance of Z , can be obtained as

$$\rho = \left(1 + \frac{U \cdot f(U) - L \cdot f(L)}{\Phi(L) - \Phi(U)} - \mu_{\text{cond}}^2 \right).$$

C.3.2 Variance Scaling for Clustered Attribute Sets. The above formulation indicates that, when we are given the truncation boundaries l and u , we can compute a variance scaling factor ρ for $\text{var}(Z|S)$. However, in the context of our problem, we are seeking $\text{var}(Z|S)$, where S is nothing but just one of the *multi-attribute* clusters obtained through clustering of the values of the attribute set $\mathcal{Z}$. This means that, we need to compute an *average* variance scaling factor, without having a priori knowledge of the multi-attribute cluster $\mathcal{S}$.

Preliminaries. Let D be a data set and let $k = |\mathcal{Z}|$ denote the number of conditioning attributes in the given conditioning attribute set, $\mathcal{Z}$. Let $D_{\mathcal{Z}} = \Pi_{\mathcal{Z}}(D)$ denote the projection of D on the conditioning attribute set and let $\bar{D}_{\mathcal{Z}}$ be the version of the projected set, such that each attribute is variance normalized; i.e.,

$$\forall Z \in \mathcal{Z} \forall d \in D_{\mathcal{Z}} \quad \bar{d}.Z = \frac{d.Z - \mu_Z}{\sqrt{\text{var}(Z)}}.$$

Given the above, let $\bar{\mathcal{S}}$ denote the set of clusters of $\bar{D}$ obtained through a K-Means-style clustering algorithm (i.e., seeking tight clusters around a centroid) and let $\bar{S} \in \bar{\mathcal{S}}$ be one of the resulting clusters. Finally, let $S \subseteq D_{\mathcal{Z}}$ be the non-normalized data cluster corresponding to $\bar{S} \in \bar{D}_{\mathcal{Z}}$.

Since the clusters are obtained on variance normalized data using a clustering algorithm that seeks tight groupings around a centroid, for the analysis we present in this subsection, we will assume that $\bar{\mathcal{S}}$ consists of (hyper-)spherical clusters – indicating that for all $\bar{S} \in \bar{\mathcal{S}}$, we have $\text{var}(\bar{S}.Z_i) = \text{var}(\bar{S}.Z_j)$ and $\text{cov}(\bar{S}.Z_i, \bar{S}.Z_j) \sim 0$ for $i \neq j$. Finally, to simplify the analysis, we will assume that for all $\bar{S}_a, \bar{S}_b \in \bar{\mathcal{S}}$, we have $\text{var}(\bar{S}_a) = \text{var}(\bar{S}_b)$. Given the above, (a) the clusters in $\bar{\mathcal{S}}$ are similarly shaped to each other and (b) each cluster is similarly shaped across all of its attributes. Therefore, in the rest of the analysis, we can simply refer to $\bar{S}.Z$, without worrying which cluster in $\bar{S} \in \bar{\mathcal{S}}$ or which attribute in $Z \in \mathcal{Z}$ we are considering.

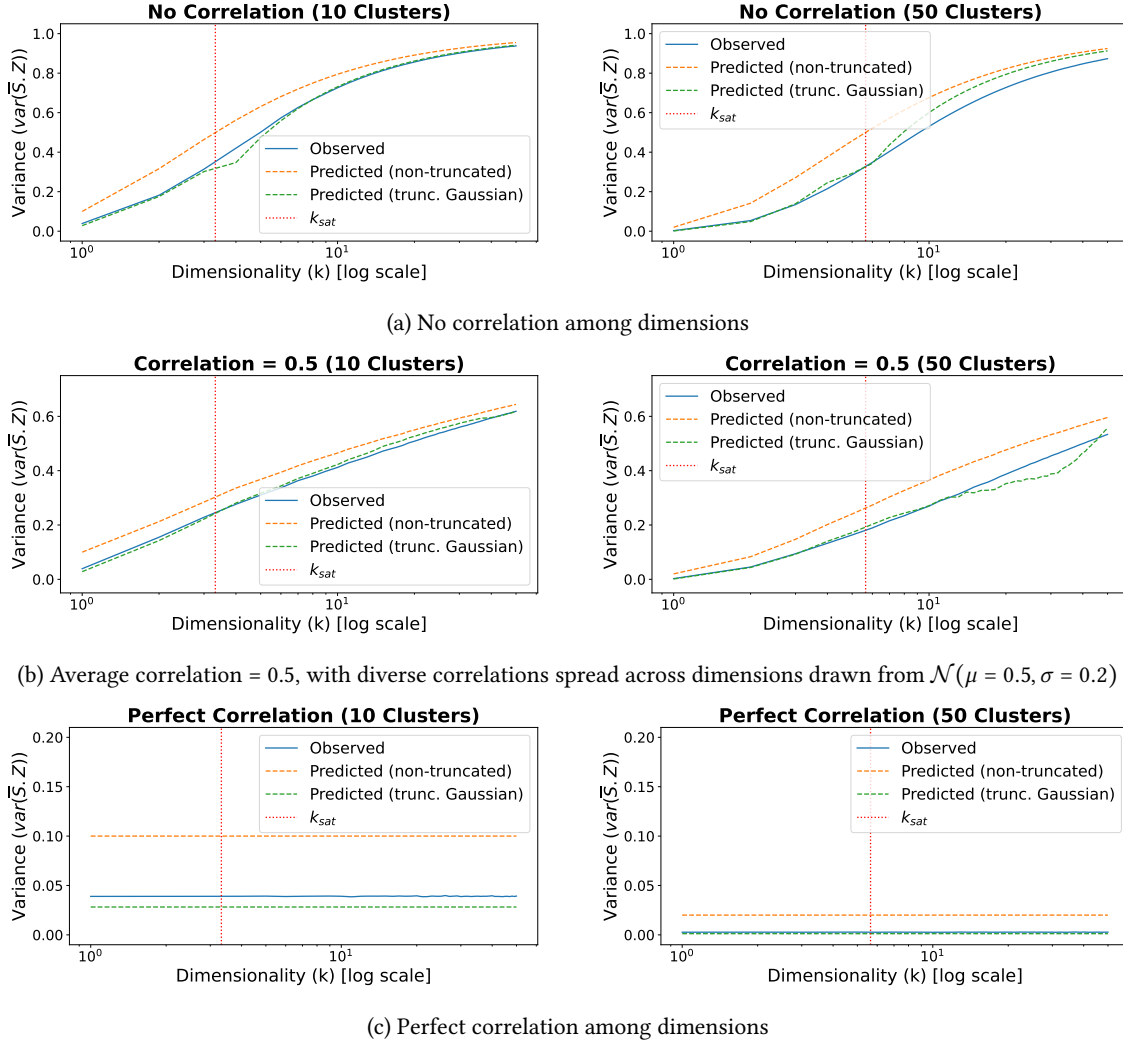


Figure 27: Predicted vs. observed $var(\bar{S}.Z)$ as a function of the number of dimensions, for 10 and 50 clusters, for data with (a) no correlation, (b) 0.5 correlation, and (c) perfect correlation among data dimensions (the threshold k_{sat} is $\log_2 10$ and $\log_2 50$ for 10 and 50 clusters, respectively) – as we see in these charts, the non-truncated approximation overestimates the variances, whereas, the proposed truncated Gaussian based model accurately predicts variances, especially in low dimensional settings, which are more likely to occur in practical skyline query settings

Impact of the Dimensionality and Data Correlation. To investigate the impact of dimensionality of the conditioning attribute set (i.e., the value of $|\mathcal{Z}|$) on the overlap between the clusters (which in turn impacts the value of $var(\bar{S}.Z)$), let us first consider the case where the data is perfectly correlated.

Perfect Correlation among Dimensions When the correlation of the data is 1.0, since all the data are on the same line, the data will behave as if $k = 1$ and clustering process will return non-overlapping partitions on that line. This would imply that we could approximate the variance as follows:

$$var(\bar{S}.Z) \sim c^{-1} \cdot var(\bar{D}_{\mathcal{Z}}.Z).$$

This approximation, however does not take into account that the clusters need to be modeled as truncated Gaussians and we need to compute the variance scaling factor using the truncated Gaussian's discussed in Section C.3.1. In particular, assuming the clustering algorithm returns c equi-width clusters, then, considering that $\sim 99\%$ of the points are within ± 3 standard deviations) of μ , assuming that the overlap between cluster boundaries are negligible, and assuming that c is an even number, for all $-\frac{c}{2} \leq i < \frac{c}{2}$, we have the following boundary pair:

$$l_i = \mu + \frac{3 \cdot \sqrt{V}}{c/2} \cdot i ; \quad u_i = \mu + \frac{3 \cdot \sqrt{V}}{c/2} \cdot (i + 1)$$

Since we are working with pre-normalized data, we have $\mu = 0$ and $V = \text{var}(\bar{D}_{\mathcal{Z}}.Z)$. Given these, we can then numerically compute $\text{var}(\bar{S}.Z)$ as the average of the variations of the (truncated Gaussian) clusters:

$$\text{var}_{PC}(\bar{S}.Z) \sim c^{-1} \cdot \sum_{i=-\frac{c}{2}}^{\frac{c}{2}-1} \text{VAR}_{trunc_Gauss}(c, i, \text{var}(\bar{D}_{\mathcal{Z}}.Z)),$$

where $\text{VAR}_{trunc_Gauss}(c, i, V)$ denotes the truncated Gaussian variance for the i^{th} segment of the Gaussian ($-\frac{c}{2} \leq i < \frac{c}{2}$) with variance V segmented into c equi-bin partitions. The case for when c is odd can be trivially extended from above.

No Correlation among Dimensions. Let us next consider the case where the data $\bar{D}_{\mathcal{Z}}$ are distributed in the $k = |\mathcal{Z}|$ dimensional space, such that $\mathcal{Z}$ are not correlated. In this case, the space will be covered by $c = |\mathcal{S}|$ Gaussians (each corresponding to a cluster). This would imply that we would have $\sim c^{\frac{1}{k}}$ layers of clusters along each dimension and that the combined variation of the $c^{\frac{1}{k}}$ clusters along a given dimension should be proportional to the variation of the data along that dimension:

$$\text{var}(\bar{S}.Z) \sim c^{-\frac{1}{k}} \cdot \text{var}(\bar{D}_{\mathcal{Z}}.Z).$$

this analysis, however, has two limitations: (a) firstly, it does not take into account that the clusters need to be modeled as truncated Gaussians and (b) does not take into account the situations where $c^{\frac{1}{k}} < 2$, i.e., there are not sufficient clusters to divide each dimension to at least two partitions. Next, we discuss how to handle these two limitations.

First, let us assume that $c^{\frac{1}{k}} \geq 2$, i.e., there are sufficient clusters to divide each dimension into two. Since we are considering the situation where the dimensions of the data are not correlated, the covariance matrix, Σ , of the data will be diagonal and, since the data has been pre-normalized, we would have $\sim c^{\frac{1}{k}}$ layers of clusters along each dimension. Therefore, we can then numerically compute $\text{var}(\bar{S}.Z)$ as the average of the variations of $c^{\frac{1}{k}}$ (truncated Gaussian) clusters:

$$\text{var}_{NC}(\bar{S}.Z) \sim c^{-\frac{1}{k}} \cdot \sum_{i=-\frac{k\sqrt{c}}{2}}^{\frac{k\sqrt{c}}{2}-1} \text{VAR}_{trunc_Gauss}(k\sqrt{c}, i, \text{var}(\bar{D}_{\mathcal{Z}}.Z)).$$

The case for when $c^{\frac{1}{k}}$ is odd can be trivially extended from above.

Next we consider when the number of clusters is not sufficient to partition each of the k dimensions; i.e., $c^{\frac{1}{k}} < 2$. In this case, necessarily, some of the dimensions will be left non-partitioned. Let this threshold beyond which some dimensions are saturated and cannot be partitioned be $k_{sat} = \log_2(c)$. Let $k_p \leq \text{round}(k_{sat})$ be the number of partitioned dimensions and k_n be the number of non-partitioned dimensions ($k_p + k_n = k$). Assuming that the K-means treats all dimensions similarly and does not over-partition certain dimensions, while under partitioning others, this means that we will have $k_p = \text{round}(k_{sat})$ dimensions partitioned into two and $k_n = k - \text{round}(k_{sat})$ dimensions left non-partitioned.

Consequently, we have

$$\begin{aligned} \text{var}_{NC}(\bar{S}.Z) &\sim k_p \cdot \frac{1}{2} \cdot \sum_{i=-\frac{1}{2}}^{\frac{1}{2}-1} \text{VAR}_{trunc_Gauss}(2, i, \text{var}(\bar{D}_{\mathcal{Z}}.Z)) \\ &\quad + k_n \cdot \text{var}(\bar{D}_{\mathcal{Z}}.Z). \end{aligned}$$

General Case. In practice, the dimensions of the data space are neither perfectly correlated nor completely independent. A naive approach to estimate $\text{var}(\bar{S}.Z)$ would be to take the correlation-weighted average of *no correlation* and *perfect correlation scenarios*; however, this approach would have two major limitations: (a) firstly, we would need to combine all pairwise correlations in the data to a single weight value and (b) since it assumes a linear relationship between the variance of the clusters and the average correlation of the data, which is incorrect in practice, it would not take properly into account clusters that need to be modeled as truncated Gaussians. Next we propose a general formulation that addresses these limitations.

Let $\mathbf{C}$ be the covariance of $\bar{D}_{\mathcal{Z}}$. We can eigen-decompose $\mathbf{C}$ to obtain two matrices, $\mathbf{U}$ and $\mathbf{C}^*$:

$$\mathbf{C} = \mathbf{U} \mathbf{C}^* \mathbf{U}^T,$$

where $\mathbf{U}^T$ is an ortho-normal transformation that transforms the data $\bar{D}_{\mathcal{Z}}$ into $\bar{D}_{\mathcal{Z}}^*$, such that all variances are concentrated on the diagonal; i.e., $\mathbf{C}^*[i, i] = \text{var}(\bar{D}_{\mathcal{Z}}^*.\mathcal{Z}_i)$, where $\mathcal{Z}_i$ is a dimension in the transform space, and for $i \neq j$, $\mathbf{C}^*[i, j] = 0$.

Since the transformation is ortho-normal it preserves distances, which implies that the K-means in the transform space and in the original space, will return the same clusters. Moreover, since the covariance matrix is diagonal in the transform space, the effect of clustering on variances will be similar to that of the *no correlation* scenario considered above. The only difference is that, in this case, different dimensions have different variances – therefore, we do not expect to have the same number of partitions along each and every dimension. Instead, we expect the number of partitions, $c_{\mathcal{Z}_i}^*$, along a given dimension, $\mathcal{Z}_i$, in the transform space, will be proportional to its variance:

$$c_{\mathcal{Z}_i}^* \propto \text{var}(\bar{D}_{\mathcal{Z}}^*.\mathcal{Z}_i).$$

More generally, for $\mathcal{Z}_i$ and $\mathcal{Z}_j$ in the transform space, we have

$$\frac{c_{\mathcal{Z}_i}^*}{c_{\mathcal{Z}_j}^*} \sim \frac{\text{var}(\bar{D}_{\mathcal{Z}}^*.\mathcal{Z}_i)}{\text{var}(\bar{D}_{\mathcal{Z}}^*.\mathcal{Z}_j)} = \frac{\mathbf{C}^*[i, i]}{\mathbf{C}^*[j, j]}$$

and, given the number, c , of data clusters, we have

$$c = \prod_{i=1}^{k'} c_{\mathcal{Z}_i}^*,$$

from which we can obtain the number of partitions for each dimension $\mathcal{Z}_i$ ($1 \leq i \leq k'$; $k' \leq k$) in the transformed space.

As before, let $\bar{\mathcal{S}}$ denote the set of clusters of $\bar{D}$ obtained through a K-Means-style clustering algorithm (i.e., seeking tight clusters around a centroid) and let $\bar{S} \in \bar{\mathcal{S}}$ be one of the resulting clusters. Since the transformation is ortho-normal (i.e., preserving distances), we will get the same clusters in the original and transformed spaces. Therefore, assuming $c_{\mathcal{Z}}^*$ is even, for the given cluster $\bar{S}$, we

can compute the variance along dimension $\bar{S}$ in the transform space as

$$var(\bar{S}.Z) \sim \frac{1}{c_Z^*} \cdot \sum_{i=-\frac{c_Z^*}{2}}^{\frac{c_Z^*}{2}-1} VAR_{trunc_Gauss}(c_Z^*, i, var(\bar{D}^*_Z.Z)).$$

The case for when c_Z^* is odd can be trivially extended from above.

Finally, we need to transform the variances back to the original space to obtain $var(\bar{S}.Z_i)$ from $var(\bar{S}.Z_i)$ for $1 \leq i \leq k$. Let $C_{\bar{S}}^*$ be the covariance matrix corresponding to the cluster $\bar{S}$ in the transform space; i.e., $C_{\bar{S}}^*[i, i] = var(\bar{S}^*_Z.Z_i)$, where Z_i is a dimension in the transform space, and for $i \neq j$, $C_{\bar{S}}^*[i, j] = 0$. We obtain the corresponding cluster covariance matrix in the original space as

$$C_{\bar{S}} = U C_{\bar{S}}^* U^T,$$

from which we can obtain the value of $var(\bar{S}.Z_i)$ as follows:

$$var(\bar{S}.Z_i) = C_{\bar{S}}[i, i].$$

Figure 27 shows the fit of the predictions by this approach to the observed standard deviations, for different numbers of clusters and dimensions and for data with different average correlations. We note that the fit is very good, especially for lower dimensionalities, which we generally expect to see in practical skyline queries.